Article

Per-Field Categorical G-Code Recoverability from Multi-Modal Sensor Embeddings on a Desktop CNC Mill: A Single-Platform Characterization Study

Stephen S. Eacuello [1,*], Romesh S. Prasad [1] and Manbir S. Sodhi [1]

[1] Department of Industrial Systems Engineering, University of Rhode Island, Kingston, RI 02881, USA
* Correspondence: seacuello@uri.edu

Version August 25, 2026 submitted to Sensors

Abstract: A cyber-physical attack can alter the G-code a CNC controller executes without altering the loaded program file, leaving the forgery invisible to file-hash checks. We characterize **per-field categorical recoverability** of G-code from a 4 Hz seven-modality sensor stream on a Bantam Tools desktop mill (TRL 4). The model is a grammar-constrained six-head Transformer decoder over a frozen denoising encoder, evaluated under five-fold file-level cross-validation. Categorical fields are recoverable. The categorical heads (command, parameter-type, sign) are teacher-forced/autoregressive invariant, at $\sim$0.89 command accuracy. Above the operation-class modal predictor, the decoder adds a +22 pp within-class command increment. This increment is a 4-of-5-fold effect (near-zero on one covariate-shift outlier), survives conditioning the null on operation class, and is reported as an *upper* bound on sensor-driven recovery. Generative reconstruction is not recoverable. Token accuracy falls 0.78 $\rightarrow$ 0.21 and digit accuracy 0.59 $\rightarrow$ 0.10 under autoregression, with whole-sequence exact-match at 0.026. The drop reflects autoregressive mode collapse, in a sensor stream where 77.7% of blocks occupy $\leq$1 sample period. On an open-vocabulary leave-one-class-out holdout, command recovery collapses to 0.213. We therefore reserve "recovery" for categorical fields; coordinate reconstruction is unsupported, with the ceiling set by source entropy and a sub-Nyquist floor (companion paper). The reported tamper operating points are **illustrative, prior-dominated, and not a validated detector**. The intended posture is forensic-support post-hoc audit within a single-platform, closed-world scope, not a real-time intrusion-detection system.

Keywords: CNC machining; G-code reconstruction; multi-modal sensor fusion; manufacturing cybersecurity; tamper detection; grammar-constrained inference; forensic audit

1. Introduction

In a Computer Numerical Control (CNC) milling machine, motors drive a cutting tool along multi-axis toolpaths specified by a G-code program, with tolerances measured in micrometers. Every cut leaves a multi-modal sensor signature in the vibration of the machine frame, the acoustic emission, the motor-current draw, and the ambient temperature and illumination. That signature is in principle informative about the instruction stream that produced it. Whether it is informative enough to *recover* the instruction stream line by line, without access to the controller, is the engineering question this paper takes up. The answer carries direct consequences for sensor-side recoverability bounds and for deployment-grade audit of contested runs.

We approach that question on a single desktop CNC milling platform (Bantam Tools desktop mill, machinable-wax workpiece, dry stepper-driven cutting, 4 Hz envelope-extracted sensor stream). Cross-platform replication on a production-class VMC is future work (Section 8.6) and a precondition for any deployment claim outside this scope. The intended application is **forensic-support post-hoc**

audit of contested runs, in which a controller-independent record of executed instructions is reviewed offline against the controller log when a run is flagged. That record complements file-hash signing and controller attestation; it is not a real-time intrusion-detection system. Within that audit posture, the per-field characterization additionally supports offline process verification and human-in-the-loop triage in increasingly networked manufacturing environments [1–6]. The triage covers a narrow subset of G-code substitutions, limited to command, axis, and motion-sign edits within the trained toolpath envelope.

Three boundaries are explicitly out of scope. First, real-time autonomous SOC tier-1 alerting is *not* a supported deployment posture on this stack (Section 7.4). Second, numeric-coordinate and feed-rate substitutions remain undetectable on the present 4-Hz sensor stack by an information-theoretic bound (Section 7.4). Third, the audit posture is bound to *in-distribution* inputs, closed-world over the V8 token vocabulary (Section 3.2) and the nine training-time operation classes (Section 5.1); novel-command and novel-operation-class inputs fall outside this envelope (LOCO collapse, Section 6.12) and are out of scope for the present system. The cost envelope a wider sensor stack could in principle catch (scrapped parts, damaged tooling, or worse) [7,8] is therefore framing for future cross-platform replication, not a claim of the present system.

Multi-modal sensor monitoring of CNC machines is a mature subfield, but its targets have been broad. Tool-wear estimation [7–11], operation-type classification [12,13], and process-monitoring tasks such as surface-quality and anomaly detection [14,15] answer session-level questions: *is the tool worn?*, *what kind of operation is running?*, *is the surface within tolerance?* Their target labels are coarse, a single value per recording or a class per file. The finer question, whether the specific G-code instruction executing at this moment can be identified from sensors alone, has been largely absent from the CNC literature. That absence persists despite direct implications for tamper detection, for the dual-use side-channel leakage of manufacturing instructions [16–21], and for root-cause analysis on contested runs.

Why CNC milling is harder than the additive-manufacturing analogue.

The closest body of work is side-channel reconstruction of toolpaths in additive manufacturing [16–18]. In that setting, Fused Deposition Modeling deposits material one axis at a time in a controlled environment, sensors can be placed freely without risk of damage, and the sensor-to-G-code mapping is comparatively low-noise. CNC milling is a substantially harder signal environment. Simultaneous multi-axis interpolation, nonlinear force-engagement relationships, structural vibration through the machine frame, and high-energy chip ejection corrupt sensor signals and physically constrain where sensors can be mounted [7,12]. The mapping is also many-to-one. Many G-code instructions produce broadly similar sensor signatures when their cutting parameters coincide, and the same instruction can be cut at different speeds, depths, or materials.

Building on prior work in our group.

The sensor encoder used in this study is a prior contribution that achieved 96.3% accuracy on nine-class CNC operation classification using a multi-modal denoising autoencoder over six distributed sensor boards [22]. That work established the encoder's representational competence at *operation-class* granularity. A companion contribution defined a hybrid grammar-based tokenization of G-code suitable for sequence models [23]. The present paper takes the encoder as a fixed feature extractor and the grammar as a vocabulary. It asks what additional information the encoder's per-window latent carries about the row-level instructions that drove the cut.

Three challenges specific to this task.

Direct G-code generation from sensor signals raises three issues that off-the-shelf sequence-to-sequence models [24–27] do not address. (i) A 64-second window straddles tens to hundreds of G-code rows, so a naive window-level target destroys row-level information. (ii) The numeric vocabulary contains thousands of bucket tokens, most absent from any one training fold. (iii) Positional metadata (window index, file identifier) is a shortcut that inflates accuracy without proving sensor-driven recovery [28,29]. The contributions below address each.

Research questions. Four research questions guide this study.

RQ1. Per-field recoverability (window-aggregated). Which components of a G-code line (command identity, axis presence, motion direction, numeric value) are recoverable from sensor data alone, and with what fidelity? At the released 4-Hz rate, 77.7% of G-code rows occupy ≤ 1 sample period (Section 8.2), so any answer is in practice a *window-aggregated row-modal recovery* claim, not a sub-sample-resolved per-row claim.

RQ2. The metadata-versus-sensor split. Does exposing positional metadata (window index, file identifier) to the decoder improve recovery, and which fields benefit if so?

RQ3. Sensor-modality contribution. Which of seven sensor groups (accelerometer, gyroscope, magnetometer, environmental, color, audio-RMS, electrical) carry the strongest instruction-level signal?

RQ4. Cross-class generalization. Does the decoder generalize to machining operation classes it never saw during training (leave-one-class-out)?

Headline findings (stated as bounds, not raw accuracy).

The central quantities of this paper are not the closed-vocabulary $\sim$0.89 command accuracy but two bounds that frame it. Above the operation-class modal predictor, the decoder adds a +22 pp class-prior-controlled within-class command increment (Section 7.1.0.2). We report this increment as an *upper* bound on sensor-driven recovery. It is robust to conditioning the null on operation class, but it is a 4-of-5-fold effect that is zero on one fold. On an open-vocabulary leave-one-class-out holdout, command recovery collapses to a floor of 0.213 (Section 6.12). End-to-end generative reconstruction is at floor (autoregressive whole-sequence exact-match 0.026). We therefore reserve "recovery" for the per-field categorical heads and treat generative coordinate reconstruction as unsupported on this stack.

Contributions.

(1) Per-row supervised target formulation. Each sensor window is mapped to the single G-code line that fires within it. This formulation yields $\sim$19,500 training samples per fold versus $\sim$300 for a window-level alternative and eliminates within-window label ambiguity.

(2) G-Code Decoder architecture. A Transformer decoder with six structured prediction heads (token, type, motion-command, parameter-type, sign, digit) operating on the frozen latent of a prior multi-modal denoising encoder [22]. Inference is grammar-constrained at the token head so that structurally invalid transitions are masked at runtime, following recent work on grammar-aware decoding [30–32].

(3) Per-axis recoverability characterization. An evaluation that decomposes G-code into per-axis fields and reports separately for each field whether (a) the axis is detected, (b) the motion direction is correct, and (c) the numeric value is recovered. The corpus supports this analysis on the X, Y, Z, and R fields; F is reported only at thin support; S, I, and J have effectively zero positive support and are not evaluable (Table 3). This separation reveals which fields are sensor-recoverable and which are not.

(4) Ablation matrix. Cross-validated ablations of (a) positional-metadata access, (b) per-row vs. full-window target formulation, (c) sequence-classifier prior, (d) vocabulary precision, (e) each of seven sensor modalities, and (f) a no-numeric retraining of the decoder (Section 6.10.2). The no-numeric retraining decomposes the autoregressive mode-collapse into a measurable numeric-desynchronization contribution and a dominant intrinsic structural-stream collapse. Methodology follows ablation-study practice in industrial neural-network research [28,29].

(5) Cross-class generalization study. A leave-one-class-out study across nine machining operation classes that stress-tests whether the decoder memorizes per-class shortcuts or generalizes to unseen toolpath strategies within the present platform/material envelope. The cross-platform / cross-material generalization question (production-VMC, cast-iron, flood-coolant) is explicitly out of scope and is the principal open direction (Section 8.6).

(6) From-scratch Transformer seq2seq baseline. We compare the headline architecture against a from-scratch Transformer seq2seq baseline (Section 6.14). The comparison scopes the architectural contribution to autoregressive deployment-mode inference, where the frozen encoder + grammar-constrained decoder + structured heads buy the accuracy lift over the baseline. The companion limit paper [33] develops the information-theoretic account of *why* the per-field recoverability pattern takes the shape characterized here (Section 2.3).

Scope statement.

Every quantitative result in this paper is conditional on a single desktop CNC platform (Bantam Tools desktop mill, stepper drive, no flood coolant), a single workpiece material class (machinable wax / soft polymer), and a single envelope-extracted 4 Hz aggregate sensor stream; the contrast with production vertical machining centers is quantified in Section 5.1 ("Platform characterization"). The present paper is therefore a single-platform characterization study, and cross-platform replication is future work (Section 8.6).

Section 2 reviews related work; Section 3 formalizes the task; Section 4 describes the architecture; Section 5 details the dataset and protocol; Section 6 addresses RQ1–RQ4 in order; Section 7 interprets the results, with the per-attack-class detectability characterization in Section 7.4; Section 8 outlines limitations and future directions.

2. Related Work

Four bodies of prior work bear on the present study: sensor-based CNC monitoring and modality fusion (Section 2.1); sequence-to-sequence modeling and grammar-constrained decoding (Section 2.2); side-channel reconstruction of manufacturing instructions (Section 2.3); and manufacturing cybersecurity and process verification (Section 2.4).

2.1. CNC Process Monitoring and Multi-Modal Sensor Fusion

Sensor-based monitoring of machining processes is a mature subfield. Early work identified force, vibration, acoustic emission, temperature, and motor current as complementary modalities for tool-condition assessment [7]. Hand-crafted features extracted via Fourier and wavelet transforms became standard practice [9,10]. Deep learning later enabled end-to-end feature learning from raw signals, reducing reliance on domain-specific feature engineering [12,13]. Convolutional networks applied to vibration spectrograms and raw multi-channel signals achieved strong results on bearing fault detection [34], surface-roughness classification [14,35], and CNC anomaly detection [15].

Multi-modal fusion strategies span early, late, and intermediate approaches [36,37]. Cross-modal attention has shown promise when modalities are complementary rather than redundant [11,38]. Motor-current and servo-drive signatures are an established adjacent modality for load and toolpath inference within this tradition [7,12]. They form one of the seven modality groups fused by the encoder used here.

Digital-twin and digital-shadow process verification is a separate, model-based line that compares observed machine state against a simulated reference of the *intended* program [40]. The present work instead reconstructs the executed instruction stream from sensors alone, without a reference model or a trusted controller. The two lines are therefore complementary and rest on orthogonal trust assumptions.

Two gaps persist relative to instruction-level recovery. First, target labels in prior work are session-level (tool worn / not worn) or coarse class labels (turning vs. milling). A review by Tsanousa *et al.* [39] confirms that the field remains dominated by these tool-condition and operation-classification tasks. Per-row instruction targets are largely absent. Second, ablation studies that progressively remove sensor groups to quantify their marginal contribution [28,29] are uncommon in CNC monitoring. Practitioners therefore lack quantitative guidance on which sensors carry instruction-level signal as distinct from operation-class signal.

2.2. Sequence-to-Sequence Modeling and Grammar-Constrained Decoding

Sequence-to-sequence neural models were established for natural-language translation [24–26]. They have since been applied to speech [27], time series [41], and robotics [42]. Long Short-Term Memory networks [43] remain widely used for sequential industrial data where streaming inference and causal ordering matter [44]. Denoising autoencoders [45] learn representations robust to input corruption, a property that matters when industrial sensors are subject to mechanical, electromagnetic, and thermal noise.

When the output sequence follows a known grammar, constrained decoding can prune structurally invalid tokens at inference time. Picard [30] pioneered incremental parser-guided decoding for text-to-SQL. Synchromesh [46] extended the idea to general code generation with a target-side constraint language. LMQL [47] formalized constrained generation as a query-language abstraction over the model. More recent work has pushed the runtime cost of grammar masking toward zero. Outlines [31] compiles regular expressions and context-free grammars into finite-state automata reused across decoding steps. XGrammar [32] achieves near-zero-overhead per-token masking for JSON and code generation. Geng *et al.* [48] demonstrate that grammar-constrained decoding can recover structured outputs from off-the-shelf pretrained models without any task-specific fine-tuning. G-code is amenable to the same treatment because instructions consist of an optional command token followed by address-value pairs subject to first-order ordering constraints [23,49].

Recent work on large-language-model code generation for additive manufacturing [50–52] addresses a related but distinct task, generating G-code from natural-language descriptions or images. None of it considers sensor-conditioned recovery.

2.3. Side-Channel Reconstruction of Manufacturing Instructions

The closest analog to this work is side-channel reconstruction of toolpaths in additive manufacturing. Al Faruque *et al.* [16] recovered Fused Deposition Modeling print geometry from acoustic emissions, achieving high agreement with the source G-code. Song *et al.* [17] showed that a smartphone microphone alone could recover sufficient information to reproduce printed objects. Chhetri *et al.* [18] extended the approach to a detection-oriented setting, identifying kinetic anomalies indicative of attacks. More recent work uses optical [21] and combined acoustic-magnetic [20] side-channels to reconstruct full part geometries from 3D printers. Sturm *et al.* [3] and Yampolskiy *et al.* [4] survey cyber-physical attack taxonomies in additive manufacturing.

The AM side-channel literature typically uses *raw acoustic emission* signals sampled at kHz–MHz rates, whereas the present work uses an *audio-RMS envelope* scalar feature sampled at 4 Hz. The audio-RMS feature retains the energy magnitude of the cutting sound but discards the spectral content that drives most prior AM-side-channel methods. The per-instruction recovery reported here is therefore not directly comparable to the AM literature and is more conservative in its sensor demands.

Because CNC milling is a harder sensor environment than additive manufacturing (Section 1), the additive-manufacturing recovery methods do not transfer directly. Kimmell *et al.* [6] demonstrated sensor-based detection of control-injection attacks in CNC machining using energy-anomaly thresholds. Their task is binary detection rather than instruction-level reconstruction. To the best of our knowledge, no prior work reports per-row G-code recovery from CNC sensor data with structured-output evaluation.

A companion limit paper [33] develops the information-theoretic account of *why* per-field recoverability stratifies the way it does, in terms of the per-head and per-toolpath source entropy of the CNC instruction stream and its 4 Hz sub-Nyquist floor. The present work treats those bounds as given and characterizes the decoder behavior against them.

2.4. Manufacturing Cybersecurity and Process Verification

Adversarial G-code substitution in PLC and CNC controllers is an established attack surface [3,4,6,53,54]. The FMNet 2024 research-needs report [2] articulates the machining-specific instruction-stream-integrity gap. NIST SP 800-82 Rev. 3 [1] provides the general operational-technology framing, specifically its continuous-monitoring guidance under the Detect function and its supply-chain-risk guidance. Against that framing, an out-of-band physics-of-record monitor is a complementary control. Its guarantee holds only insofar as the sensor path itself is trustworthy (Section 7.4). Reconstructing the actually-executed instruction stream from external sensors, independent of the controller, provides an integrity-checking baseline that cannot be subverted by controller-resident malware. The present work characterizes which components of that instruction stream are realistically reconstructable on present-day sensor hardware, and which require additional sensor coverage or richer training data to recover.

Positioning.

A comparison table (Supplementary Materials Section S39) positions this study against representative prior work in sensor-based CNC monitoring and manufacturing-instruction side-channel recovery. To our knowledge this is the first per-instruction, per-field categorical recovery of CNC *milling* G-code from a multi-modal sensor stream. Prior CNC monitoring reports session-level or operation-class granularity, and additive-manufacturing side-channels report geometric-toolpath granularity. The file-level cross-validation with leave-one-class-out generalization testing is the evaluation methodology supporting that claim, not a co-equal novelty leg.

3. Problem Formulation

3.1. Task Definition

Let $W \in \mathbb{R}^{T \times C}$ denote a sensor window of T samples at C channels, recorded over a fixed window length aligned to a CNC milling operation. Let g denote the G-code line that was executing within W. The task is to learn a mapping

$$f_\theta : W \mapsto \hat{g}, \qquad \hat{g} = (\hat{t}_1, \hat{t}_2, \ldots, \hat{t}_L),$$

where $\hat{g}$ is a sequence of L G-code tokens drawn from a vocabulary $\mathcal{V}$. The training objective minimizes a multi-head cross-entropy loss with the auxiliary structured heads described in Section 4.3, plus an end-of-sequence token. At inference, the decoder generates tokens autoregressively under a grammar mask that zeros structurally invalid transitions in the legacy-token head.

3.2. Vocabulary and Token Types

Following CNC-shop convention, we call each G-code line a *block*. Blocks divide into *rapid moves* (`G0`, traverse without cutting) and *cutting moves* (`G1`/`G2`/`G3`, linear and arc feeds). We tokenize both block types identically, using a hybrid scheme that disentangles structural primitives from numeric values [23,49]. The vocabulary contains four token classes:

- **SPECIAL tokens**: `PAD`, `BOS`, `EOS`, `UNK`, `MASK`.
- **COMMAND tokens**: literal motion or machine commands (`G0`, `G1`, `G2`, `G3`, `M3`, `M5`, `M30`, . . .).
- **PARAM tokens**: address letters that introduce an axis or rate parameter (`X`, `Y`, `Z`, `I`, `J`, `K`, `R`, `F`, `S`, `P`).
- **NUMERIC tokens**: bucketed numeric values, one bucket per (axis, magnitude) pair. The full vocabulary contains 1,048 `NUM_X` buckets, 1,086 `NUM_Y` buckets, and 231 `NUM_Z` buckets, with smaller bucket sets for the remaining axes. The total vocabulary size is 2,418.

A typical corpus line such as `G1 X1.4919 Y1.4848 F22.` becomes the token sequence `BOS G1 X NUM_X_1492 Y NUM_Y_1485 F NUM_F_22 EOS`. The bucketization discretizes numeric values to a fixed precision. The per-position digit head described in Section 4.3 provides an auxiliary regression-style signal to mitigate the resulting discretization loss.

3.3. Per-Row Target Formulation

A naive supervised target assigns one G-code label per sensor window. In the present corpus, a 256-sample window at 4 Hz spans 64 seconds and contains an average of 60 distinct G-code rows (median 29, max 237). A window-level target therefore aggregates information across these rows and discards row-level distinctions, such as that between an `X` positive move and an `X` negative move within the same window. We instead emit one (W, g_r) training pair per G-code row r that fires within window W, where g_r is the token sequence for that single row. This per-row formulation:

- Removes within-window label ambiguity. Each (W, g_r) pair has exactly one correct target.
- Increases the training-sample count by an order of magnitude (∼19,500 per fold versus ∼300 for the window-level alternative).
- Decouples the supervised target from the windowing schedule, preventing the model from inferring labels via positional shortcuts on `(window_index, source_file)`.

A complementary *full-window* formulation, evaluated as an ablation (Section 6.10.3), retains one sample per window but uses a multi-line target consisting of all G-code rows that fired within the window, concatenated in temporal order. This formulation has fewer training samples per fold but a richer per-sample supervisory signal. It tests whether row-level autoregressive context (the previous row's tokens) closes any per-row recovery gaps.

3.4. Evaluation Metrics

We report seven complementary metrics:

1. **Token accuracy**: the proportion of non-padding token positions in the generated sequence whose argmax matches the target. This is the headline metric for sequence quality.
2. **Sequence accuracy**: the proportion of test samples for which *every* non-padding token matches the target. This is a strict exact-match metric.
3. **Type accuracy**: per-token classification accuracy on the 4-way SPECIAL/COMMAND/PARAM/NUMERIC type head. It diagnoses whether the model is in the right structural mode at each step.
4. **Command accuracy**: per-token classification accuracy on the 6-way motion-command head, evaluated only at positions where the target carries a COMMAND token.
5. **Parameter-type accuracy**: per-token classification accuracy on the 10-way axis-letter head, evaluated at PARAM positions.
6. **Sign accuracy**: per-position classification accuracy on the 3-way (positive, negative, absent) motion-sign head, evaluated at the relevant axis positions.
7. **Numeric (digit) accuracy**: per-digit-position correctness pooled across all six digit positions in NUM tokens. The companion regression measure for numeric values is the mean absolute error (MAE) of the reconstructed coordinate.

Macro precision, recall, and F1 are computed for each classification head, using scikit-learn [55] on the per-token confusion matrix.

Teacher-forced vs. autoregressive evaluation.

For any autoregressive sequence model, two evaluation regimes give materially different numbers and reflect different deployment scenarios.

Teacher-forced (TF) evaluation supplies the ground-truth previous token at every position during the forward pass and measures per-position classification accuracy under that ideal input. It is the standard objective during training and the sweep-time metric used during hyperparameter selection.

Autoregressive (AR) evaluation requires the model to consume its own previous predictions at each step, with no access to ground truth. It measures end-to-end generation quality under the realistic deployment condition in which no controller log is available to anchor the sequence.

The categorical heads in our decoder (type, command, parameter-type, sign) are independent classifiers on the encoder memory and therefore report identical accuracies under TF and AR. The autoregressive-token heads (legacy-token, digit-value, and the derived sequence and numeric metrics) report substantially lower accuracies under AR than under TF, because exposure-bias errors compound along the generated sequence.

The headline result table reports both regimes for the autoregressive heads, since sweep-time selection and deployment call for different regimes; for the categorical heads a single number suffices because the regime distinction does not apply.

3.5. Threat Model

We state the threat model up front so that every recoverability result that follows can be read against it. Section 7.4 treats the operating-point and adaptive-adversary detail.

Trusted elements.

The multi-modal sensor stack and its acquisition pipeline are trusted: sensors are physically secured, the telemetry path is integrity-protected, the recorded sensor stream faithfully represents the physical machine state, and the decoder model and verification host are not co-located with the controller. These properties are operational requirements on the deployment, not features implemented in the present prototype. The laboratory acquisition pipeline used to generate the corpus uses unauthenticated USB sensor links and is not itself a deployment-grade trust boundary. Trusting the sensor path is realistic for a process-monitoring deployment in which sensors and analysis live outside the CNC controller's trust boundary, but it is a non-trivial operational assumption (Section 7.4, "Sensor-side adversary"). A controller-resident adversary with bus access to shared sensor infrastructure is out of scope, as are sensor-side wire-tap, EMI injection, packet replay, and calibration-spoofing attacks. We additionally assume that the operator who initiates each run is authenticated to the verification host and that the program loaded on the controller is the same artifact whose hash was audited (e.g., via a controller-side load-time attestation receipt [53] bound to the operator session). An insider-threat operator who loads an unaudited program after attestation is a residual adversary the present scope does not address.

Untrusted elements.

The CNC controller, its firmware, and any G-code dispatch path between controller and machine are untrusted. The adversary may have read/write access to the controller's program memory, may inject substituted G-code, may modify cutting parameters mid-execution, or may replay an earlier benign program while a different program is intended.

Attack classes considered.

We consider five attack classes: (i) *block substitution* (e.g., `G1 X1.5` becomes `G1 X2.5`; substituted blocks may differ in command identity, axis, sign, or numeric value); (ii) *parameter tampering*, in which the program is structurally preserved but cutting parameters (feed, spindle speed, depth of cut) are altered; (iii) *replay* of a previously-executed valid program while a different program was commanded; (iv) *block skip or duplicate* of executed blocks while the controller log reports nominal execution; and (v) *upstream CAM / post-processor compromise*, in which the toolpath is corrupted before the operator-signed G-code file is produced. Class (v) is enumerated for completeness but is out of scope for the present detector, which agrees with an already-corrupted controller stream (Section 7.4). Per-attack-class detectability is reported in Table 9 (Section 7.4). In the MITRE ATT&CK for ICS taxonomy [56] these classes map respectively to T0843 / T0889 (Program Download / Modify

Program), T0836 / T0831 (Modify Parameter / Manipulation of Control), T0856 (Spoof Reporting Message), T0855 (Unauthorized Command Message), and T0862 (Supply Chain Compromise). The decoder occupies the *Detect* function of NIST CSF 2.0 (continuous-monitoring category DE.CM) [57], complementing prevention controls (file signing, attestation) rather than replacing them. *Protect*, *Respond*, and *Recover* remain out of scope.

Adversary model: orthogonal defenses.

The sensor decoder is intended to run in parallel with the cheaper orthogonal defenses (file-hash / cryptographic signature on the G-code file; TCB attestation on the controller), not in place of them. Each defense covers attack classes the others cannot. The sensor decoder's residual value is the attack subset that survives a signed-G-code audit and defeats controller attestation that does not extend to runtime memory: runtime substitution, selective skip / duplicate, and wrong-workpiece / wrong-fixturing. Within that subset, the decoder is further bounded by the closed-vocabulary in-distribution regime under which it is trained (Section 5.3) and by the per-field recoverability characterized below. The full three-layer defense-in-depth comparison is given in the positioning table (Supplementary Materials Section S39) and Section 7.4.

4. Method

4.1. System Overview

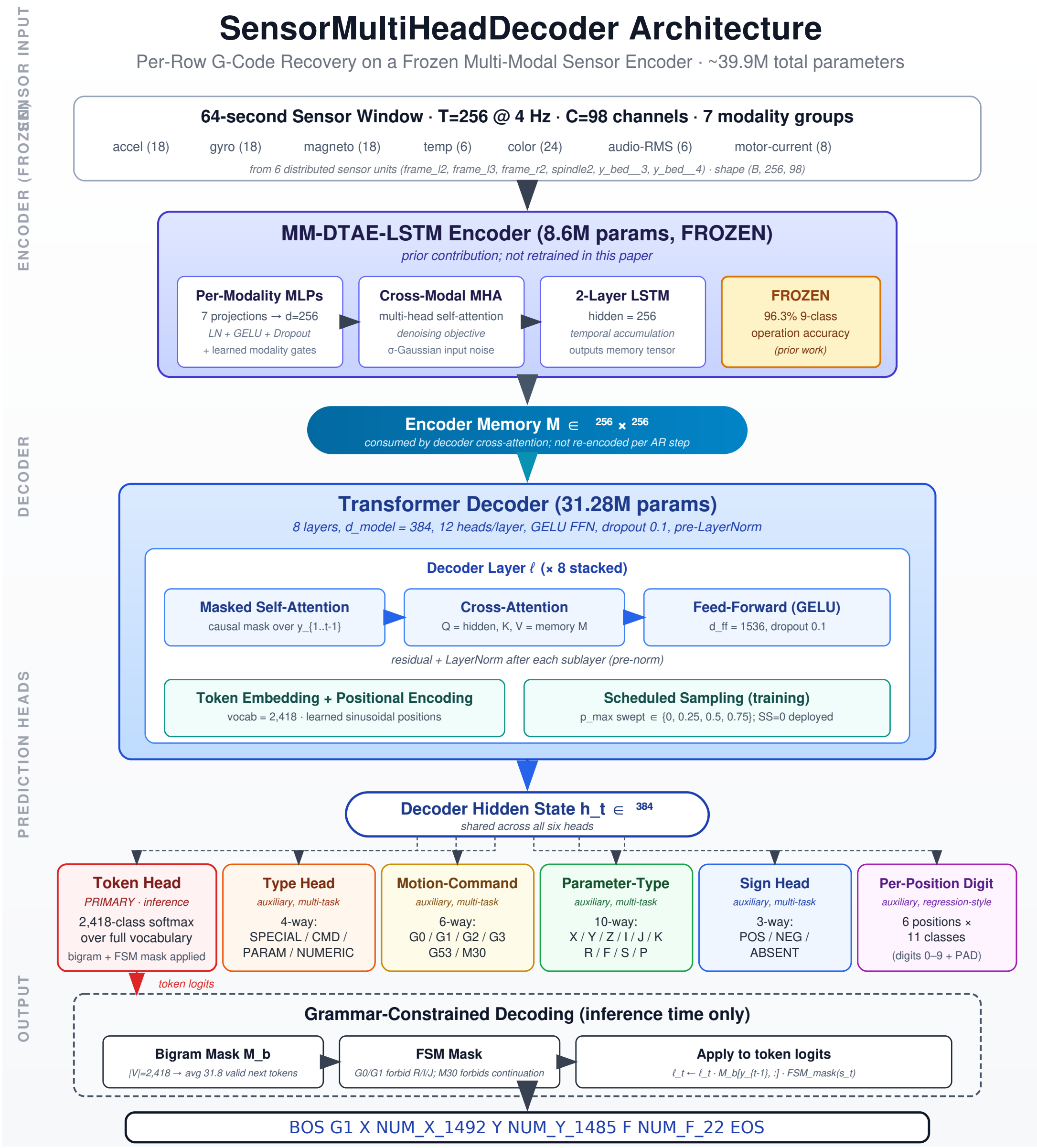

Figure 1. System overview. A 64-second sensor window (256 samples $\times$ 98 channels across seven modality groups) is mapped by the frozen MM-DTAE-LSTM encoder (8.6 M parameters) to a 256×256 memory tensor. An 8-layer Transformer decoder (d_{model}=384, 12 heads) consumes this memory via cross-attention and generates tokens autoregressively. Six prediction heads share the decoder's hidden state and provide structured output (token, type, motion-command, parameter-type, sign, per-position digit). A static grammar mask zeros structurally invalid token transitions in the legacy-token softmax at inference time.

The system has three stages (Figure 1). A multi-modal sensor encoder maps a (256-sample, 98-channel) sensor window into a (256-step, 256-dim) memory sequence. A Transformer decoder consumes this memory via cross-attention and generates a token sequence representing the G-code line that was executing within the window. A grammar mask zeros structurally invalid transitions in the decoder's softmax distribution at inference time. The encoder is frozen for all experiments; its training is described in a separate contribution [22] and is summarized here only insofar as it influences the decoder.

4.2. Sensor Encoder (Frozen)

The encoder is a multi-modal denoising autoencoder coupled to a temporal LSTM (MM-DTAE-LSTM). Its input is seven modality groups: accelerometer (3-axis, 6 boards = 18 channels), gyroscope (18), magnetometer (18), environmental temperature (6), ambient color (24), audio RMS (6), and motor current (8), for a total of $C = 98$ channels. A per-modality projection layer maps each modality into a shared $d = 256$ space. Learned modality-weighting gates [36,37] produce a fused representation, and a multi-head self-attention stack [26] captures cross-channel interactions. A denoising reconstruction objective [45] encourages noise-robust representations. A final two-layer LSTM [43,44] produces the temporal memory consumed by the decoder. The full encoder has 8.6 M parameters and was trained for 30 epochs on the same file-level 5-fold splits used here, reaching 96.3% test-set accuracy on the same nine-class operation-type task [22]. It is frozen for all decoder experiments and serves as a fixed feature extractor. A representative test window, showing the seven modality groups aligned in time against the G-code rows that fired within it, is reproduced in Supplementary Section S13.1 (Figure S16) as the visual analogue of the task formulation.

4.3. Decoder Architecture

G-Code Decoder is an 8-layer Transformer decoder with $d_{\text{model}} = 384$ and $h = 12$ attention heads per layer. Each decoder layer consists of (a) masked multi-head self-attention over the partial output sequence, (b) multi-head cross-attention over the encoder memory, (c) a position-wise feed-forward network with GELU activation, and (d) residual connections with pre-layer normalization [58]. Dropout [59] of 0.1 is applied after each sub-layer. The decoder has 31.28 M trainable parameters at the headline configuration. The Transformer stack accounts for 18.9 M of these; the remainder is the grammar-mask and sensor-prior machinery, the six output heads, the embedding, and the positional encodings (component breakdown in Supplementary Section S19.8). With the frozen 8.6 M encoder, the total system size is $\approx$39.9 M parameters.

4.3.1. Multi-Head Structured Output

Six prediction heads share the decoder's final hidden state at each output position. Each head is a linear projection followed by a softmax (or a six-position softmax in the case of the digit head). The heads are trained jointly with a weighted cross-entropy loss:

$$\mathcal{L}_{\text{decoder}} = \lambda_{\text{tok}}\,\mathcal{L}_{\text{tok}} + \lambda_{\text{type}}\,\mathcal{L}_{\text{type}} + \lambda_{\text{cmd}}\,\mathcal{L}_{\text{cmd}} + \lambda_{\text{param}}\,\mathcal{L}_{\text{param}} + \lambda_{\text{digit}}\,\mathcal{L}_{\text{digit}} + \lambda_{\text{sign}}\,\mathcal{L}_{\text{sign}}.$$

Default weights are $\lambda_{\text{tok}} = 3.0$ and all auxiliary $\lambda = 1.0$. The auxiliary heads serve two purposes: they regularize the shared representation toward token-type-aware features, and they expose structural prediction errors that would otherwise be hidden inside aggregate token accuracy. The $\lambda_{\text{tok}} = 3.0$ default was carried forward from a Stage-1 hyperparameter sweep cell (Section 4.6) on a coarse $\lambda_{\text{tok}} \in \{1, 3, 5\}$ grid. At $\lambda_{\text{tok}} = 1$ the token head was under-trained relative to the auxiliary heads (TF token validation degraded $\sim$4 pp on fold-1); at $\lambda_{\text{tok}} = 5$ the token head was over-weighted at the cost of the categorical heads (parameter-type and sign each lost $\sim$1 pp on fold-1 validation); $\lambda_{\text{tok}} = 3$ was the validation-best compromise. This is a fold-1-only sensitivity check, not a 5-fold sweep, and we record the value as a heuristic anchor rather than an optimized hyperparameter.

Legacy token head.

A vocabulary-sized softmax (2,418 classes) operating on the decoder's final hidden state. This head is the primary inference output: at each timestep, the next token is drawn from this distribution. A grammar mask (Section 4.3.2) zeros structurally invalid transitions before the softmax.

Type head.

A 4-way classifier over {SPECIAL, COMMAND, PARAM, NUMERIC} on each output position, trained against the type of the target token at that position. It is the coarsest classification head and the easiest task, so its accuracy is an upper bound on the structural sanity of the decoded sequence.

Motion-command head.

A 6-way classifier over {G0, G1, G2, G3, G53, M30}, evaluated against the target's command token at COMMAND positions. It distinguishes rapid moves (G0), linear cuts (G1), clockwise/counter-clockwise arcs (G2/G3), the work-coordinate-cancel command (G53), and program-end (M30). G53 and M30 have very small support ($n \approx 6$ per fold, ≤ 30 pooled; Table 2), so the per-class F1 for these classes is underpowered (Section 8.3).

Parameter-type head.

A 10-way classifier over {X, Y, Z, I, J, K, R, F, S, P}, evaluated against the target's address letter at PARAM positions. On the present corpus only X, Y, Z, R, and F have positive support in the test splits (per-field frequencies in Table 3). The remaining trained labels (I, J, K, S, P) do not appear in the test data and therefore have no per-class entries in the parameter-type breakdown (Supplementary Materials Section S39).

Sign head.

A 3-way classifier over {POSITIVE, NEGATIVE, ABSENT} that predicts the sign of the most recent axis-value. It provides direct supervision on motion direction independently of the discretized numeric value.

Per-position digit head.

A six-position 11-way classifier (digits 0–9 plus PAD) that reconstructs the numeric value of each NUM token digit-by-digit. The six positions cover the precision range observed in the corpus (typically $0-2XXXX$ for an X or Y coordinate). Decomposing numeric reconstruction by digit position enables the per-position recoverability analysis of Section 6.4 and provides a regression-style auxiliary signal that complements the discretized NUM-token head.

4.3.2. Grammar-Constrained Decoding

G-code follows a regular grammar: a line starts with an optional command (G/M) token; each address letter is followed by exactly one numeric value; certain commands (e.g. G2, G3) require specific subsequent fields. We encode the local part of this grammar as a static $|V| \times |V|$ binary transition mask and apply it to the legacy-token logits at each inference step. The mask zeros structurally invalid transitions: BOS must be followed by a command or PARAM letter; a PARAM letter must be followed by its matching NUM token; a NUM token must be followed by another PARAM letter, a command, or EOS. On average the mask reduces the valid next-token set from $|V| = 2{,}418$ to 31.8 tokens, a reduction that improves both calibration and exact-match accuracy. Implementation follows the runtime grammar-mask pattern of Picard [30], Synchromesh [46], LMQL [47], Outlines [31], XGrammar [32], and Geng *et al.* [48] (reviewed in Section 2.2).

Scope and limitations of the local mask.

The bigram mask above does not encode command-state-dependent rules of the form "the active command is G0, therefore the next address letter must not be R" (R is an arc-radius parameter valid only with G2/G3). Such rules require a finite-state grammar (FSM) over a multi-token history. A separate inference-time FSM layer tracks the most recently emitted move-command as state, forbids R/I/J after G0/G1 and any command/PARAM after M30, and zeros those transitions in the legacy-token logits before the softmax. The layer is applied at autoregressive inference only; teacher-forced training is unaffected because the targets already encode correct structure. Its full state specification ($\mathcal{S} = \{s_0, s_{G0}, s_{G1}, s_{G2}, s_{G3}, s_{M30}, s_{other}\}$ with per-state forbidden-PARAM sets) is in Supplementary Materials Section S35.

FSM ablation.

Across ∼167,400 generated tokens per configuration, the bigram-only mask permits G0/G1 to emit R/I/J, producing 329 V1 violations affecting 31% of baseline samples and 333 affecting 25% of with-shortcuts samples. With the FSM active, V1 violations drop to 0 on baseline (100% reduction) and 11 on with-shortcuts (97% reduction); the residual 11 are G53/G55/G17/G90/G94 followed by R, outside the present FSM's scope. The 5-fold accuracy cost is within fold-to-fold noise ($-0.27/-0.41$ pp baseline token/numeric, $-0.04/+0.01$ pp with-shortcuts; paired-t and Wilcoxon all $p > 0.4$). The FSM therefore eliminates the dominant grammar-violation class at no measurable accuracy cost, and we adopt it as the default deployment-time decoding configuration. The full violation counts and paired-test statistics are in Supplementary Materials Section S35.

Forward-pass pseudocode.

The deployment-time grammar-masked autoregressive forward pass that produces the token sequence reported throughout this paper is given as pseudocode in Supplementary Materials Section S39. The bigram mask and the finite-state-machine layer are both applied to the legacy-token logits before the softmax. The structured heads (type, command, parameter-type, sign, digit) read the same hidden state but do not consume the autoregressive token stream.

4.3.3. Scheduled Sampling

The decoder is trained with teacher forcing as default. Optionally a scheduled-sampling regime [60] is applied: with probability p that linearly increases from 0 to a tuned maximum $p_{\max}$ over training, each input position is replaced with the model's own previous prediction rather than the ground-truth target. This reduces exposure bias between training and inference. The maximum $p_{\max}$ was chosen by the hyperparameter sweep (Section 4.6) on its composite selection score, a weighted geometric mean combining fold-1 validation token accuracy with held-out test command/numeric/sequence accuracy. The values tested were $p_{\max} \in \{0.1, 0.2, 0.5, 1.0\}$ against the $p_{\max} = 0$ teacher-forced default. The composite-best and headline value is $p_{\max} = 0.5$, which lifted command accuracy by $+17$ pp over $p_{\max} = 0$ at the cost of optimization stability at smaller numeric vocabularies (Section 6.10.2 body, where the Design-B-style runs use $p_{\max} = 0$ for this reason). On validation token accuracy alone, a cross-window-pointer cell ranked marginally higher; this selection therefore touches held-out test metrics and is a configuration choice, not an independent generalization estimate.

4.4. Training Configuration

The decoder is optimized with AdamW [61] ($\eta = 5 \times 10^{-5}$, weight decay 0.05) for up to 300 epochs with early stopping on validation token accuracy (patience 75). The first 10 epochs use a linear warm-up. Batch size is 64, mini-batches are shuffled per epoch, and gradient clipping is applied at norm 1.0. The best checkpoint by validation token accuracy is retained.

The baseline configuration trains the decoder *without* positional metadata: `window_index`, `total_windows`, and `source_file` identifiers are not exposed to the decoder. Section 6.10.1 ablates this choice. The 5-fold training/validation loss and token-accuracy trajectories (monotonic decrease; validation plateaus near epoch 80–100, well within the patience-75 budget) are in Supplementary Section S13.2 (Figure S17).

4.5. Encoder Information-Content Probe

Because the encoder is frozen, the decoder is bounded above by what its representation carries. We characterize that bound with MLP probes on the mean-pooled encoder memory: for each of (command, axis-presence, motion-sign, operation-type) a two-hidden-layer MLP (hidden 256, ReLU, dropout 0.2) is trained on train, selected on val, scored on test (MSE→RMSE for continuous numeric targets). Qualitatively, the mean-pooled memory clusters tightly by operation class. The nonlinear t-SNE projection is in Supplementary Section S13.3 (Figure S18); the linear analogue is the PCA reproduced in Supplementary Materials Section S39.

Linear structure.

PCA on the same memory matrix (549 × 256; Supplementary Materials Section S39) shows it is strongly low-rank: 9/24/40 PCs capture 80/95/99% of variance, and the first two PCs already linearly separate the operation classes. The class-discriminative signal thus lives in a low-dimensional, linearly-accessible subspace, while fine per-row numeric detail must live in the low-energy tail and is poorly preserved. This is the representational counterpart of the numeric bottleneck (Section 6.4) and of the class-prior confound (Section 7.1.0.2).

The probe results (`audit/encoder_probe_v8.json`) give three facts that bound the rest of the paper. First, *window-level signal exists*: operation type 0.846, axis presence ∼0.87–0.93 (X/Y/Z), motion sign 0.80–0.86. Second, *per-row disambiguation does not*: a window holds a median of 29 rows (max 237) but one mean-pooled vector, so probe accuracy is capped near the modal-row-per-window ceiling (the probe itself reaches command 0.77; the modal-row ceiling is ≈0.85). This formulation gap motivates the full-window comparison (Section 6.10.3) and underlies the class-prior confound (Section 7.1.0.2). Third, *numeric precision is lost*: X-value RMSE 0.97 approaches the within-window range (∼3.0 normalized units), Y is 0.80, and Z is 0.10, so the encoder retains a window-level mean, not per-row precision. Row-level encoder pre-training is the natural remedy (Section 8).

4.6. Hyperparameter Sweep

The training configuration was selected through a three-stage hyperparameter sweep on fold 1. **Stage 1** (8 cells) explored learning rate, batch size, model dimension, and loss-weighting on a coarse grid. **Stage 2** (12 cells) refined around the Stage-1 winner by varying training techniques: curriculum learning, scheduled sampling, label smoothing [62], dropout, layer depth, and focal loss [63]. **Stage 3** (10 cells) tested architectural variants: encoder fine-tuning (the last-LSTM-layer-only cell, the encoder-bottleneck probe), pointer-network output, regression-head numeric output, cross-window attention, multi-seed variance, and alternative encoder configurations. The composite winner across all 30 cells (selected on a weighted-geometric-mean score) was used for the 5-fold experiments of Section 6. The Stage-3 encoder-fine-tune pilot produced no meaningful lift (fold-1 command ∼0.78 vs. frozen-encoder ∼0.78) at ∼2× cost and was not promoted; it grounds the encoder-bottleneck claim of Section 4.5 as a measured-if-underpowered result. The sweep also ran on fold 1, the covariate-shift outlier of Section 8.3.0.1, so the configuration *rank* should be read as fold-1-validation-best rather than cross-fold-rank-stable; the headline numbers are still computed on all five folds. Both caveats are detailed in Supplementary Materials Section S38.

4.7. Architecture-Component Ablation Summary

Five architectural and training components of the decoder are ablated empirically, each pinned to the result section that reports its 5-fold effect. Supplementary Table S1 (Supplementary Section S3) consolidates the findings, one row per component, with citations to the in-body subsections that report each experiment in full.

The six prediction heads themselves are not ablated independently. The four auxiliary heads (type, parameter-type, sign, digit) share the same encoder-memory feature as the legacy-token head and serve as multi-task regularizers, so isolating each would require separate retrains. The categorical-vs-numeric factorization Section 6.2 establishes is the de facto head-level decomposition: the categorical heads are TF-AR-invariant (Section 3.4), so their effect is structural rather than dependent on autoregressive coupling.

5. Experimental Setup

5.1. Dataset

The corpus consists of 120 aligned CSV files recorded from a Bantam Tools desktop CNC milling machine [22]. Six distributed multi-sensor boards (`frame_l2`, `frame_l3`, `frame_r2`, `spindle2`, `y_bed__3`, `y_bed__4`) sample synchronously at 4 Hz. The double-underscore form matches the column-prefix convention in the released aligned CSVs. Each file carries:

- Accelerometer (3 axes $\times$ 6 boards = 18 channels);
- Gyroscope (18 channels);
- Magnetometer (18 channels);
- Ambient color sensing (RGBA $\times$ 6 boards = 24 channels);
- Environmental temperature (6 channels);
- Audio root-mean-square (6 channels);
- Motor-current draw (8 channels);

plus an aligned per-sample G-code line. We exclude proximity and pressure channels following the consistency audit reported in the encoder paper [22], retaining $C = 98$ sensor channels.

Sensor mounting and envelope-extraction pipeline.

The six multi-sensor boards are rigidly bolted to the left/right machine-frame uprights (`frame_l2`, `frame_l3`, `frame_r2`), adjacent to the spindle housing (`spindle2`), and on the moving Y-bed (`y_bed__3`, `y_bed__4`). Each board carries an identical 15-channel inventory, and an additional 8 controller-side electrical channels are recorded separately. All channels are read at factory-rated sensitivities and envelope-extracted to the released 4 Hz rate by a single-pole IIR rectify-and-low-pass filter at $f_c = 2$ Hz, decimated last-in-window and clock-aligned across boards. The extraction trades the kHz-band content of classical acoustic-emission monitoring [7] for an ingestion-friendly $\sim$ 1.5 kB/s envelope. The firmware pipeline is released so that a reproducer can regenerate the aligned CSVs from the raw stream. Full mounting/calibration detail, the per-channel inventory, and the firmware-pipeline steps are in Supplementary Materials Section S36. Chip-level part numbers and the electrical-channel mapping are in the encoder paper [22]. The LOMO/LOSO findings (Section 6.11.1, Supplementary Section S5) should be read against that inventory.

The dataset spans nine machining operation classes: `adaptive`, `adaptive150025`, `face`, `face150025`, `pocket`, `pocket150025`, `damageadaptive`, `damageface`, `damagepocket`. The `150025` suffix denotes a variant feed/spindle parameter setting of 150 mm/min feed at 25,000 RPM spindle, holding all other CAM parameters fixed. The `damage` prefix is assigned by an operational rule rather than a continuous surface-roughness threshold. A run is labelled `damage*` when executed with a tool deliberately worn or chipped prior to the cut, with the visible cutting-edge defect recorded in the run log, regardless of post-run inspection. The nine classes therefore cover three motion strategies (face, pocket, adaptive) crossed with a clean baseline, the 150025 parameter variant, and the damaged-tool condition variant. The six clean and parameter-variant classes are approximately balanced at 15–20 files each; the three damaged-tool classes are smaller at 4–5 files each. The per-class file inventory, distinct G-code-line counts, and aligned-sample counts are released in `audit/operation_class_inventory.json`.

Platform characterization and generalization risk.

The host platform is a Bantam Tools desktop CNC milling machine: cast-aluminum frame, ER11 collet spindle nominally to $\sim$ 28,000 RPM, stepper-driven 3-axis gantry, no flood coolant, and an RS274/NGC-compatible dialect. Every run cuts a single machinable-wax workpiece dry with a single carbide end-mill class. This platform sits at the low end of the manufacturing CNC spectrum. Production VMCs of the Haas/DMG/Okuma class, with cast-iron base, BT30/BT40 toolholders, servo drives, and flood coolant, differ in vibration amplitude by 1–2 orders of magnitude, and further in spectral content, acoustic emission, and motor-current signature (closed-loop servo torque vs. open-loop stepper magnitude). The detailed VMC contrast and the per-regime literature [7,8,12] are restated in the scope-bounding limitation of Section 8.1. The present corpus therefore characterizes the recoverability physics of a single low-amplitude, dry, stepper-driven platform. Whether the recoverability factorization transfers to a production VMC is the cross-platform open question of Section 8.6, not a claim the present corpus can settle.

5.2. Preprocessing and Windowing

Sliding sensor windows of $T = 256$ samples (64 seconds at 4 Hz) at stride 64 yield $\sim$1,500 windows per fold. Each window is centered on a region of continuous cutting and aligned to its overlapping G-code rows. The per-row target formulation (Section 3.3) emits one (W, g_r) pair per G-code row r that fires within window W. This yields $\sim$19,500 training pairs, $\sim$7,000 validation pairs, and $\sim$7,500 test pairs per fold. At 4 Hz across the 98 retained channels in `float32`, the on-line ingestion budget is $\sim$1.5 kB/s, equivalent to $\sim$0.13 GB per machine per 24-hour shift uncompressed, well within standard plant-IT storage tiers.

Sampling-rate context.

The 4 Hz sensor sampling rate constrains what is and is not recoverable. A row-execution-duration audit over the 120-file aligned corpus (21,541 row-execution instances; `audit/gcode_row_durations.json`) shows that the median G-code row executes in a single 0.25 s sample and that 77.7% of row-execution instances occupy $\leq$ 1 sample period; conventional acoustic-emission and vibration tool-condition monitoring samples at $\geq$ 10 kHz [7]. Every recoverability result below should be read in that light; the full physical-aliasing discussion is in Section 8.2.

5.3. Five-Fold File-Level Stratified Cross-Validation

We use file-level stratified five-fold cross-validation to prevent within-file leakage. Each operation class is divided across folds in proportion to its frequency, so each fold contains representative coverage of all nine classes. Within a fold, the train and validation splits come from the same fold partition and the test split is the held-out partition.

Closed-vocabulary evaluation regime (load-bearing).

We then run a G-code-line *coverage repair* step (Supplementary Materials Section S20). Any G-code line that appears in the test split but not in the training partner triggers a file swap between the partners, until every test line also appears in training. This step does not remove a confound. It *constructs* a deliberate closed-vocabulary evaluation regime in which every test line has been seen verbatim during training of the same fold (97.6% of distinct test lines and 99.0% of test line-instances; Section 8.1). The 5-fold headline numbers must therefore be read as a within-vocabulary classification / near-retrieval result, *not* as open-vocabulary instruction recovery. The matched open-vocabulary number is the leave-one-class-out (LOCO) command accuracy of 0.213 ± 0.191 (Section 6.12); the two should be read side by side throughout this paper.

Within-fold non-independence.

Preprocessing uses overlapping sensor windows (window 256, stride 64, i.e., 75% overlap between adjacent windows from the same file), so adjacent windows within a single file are not statistically independent. File-level disjointness across folds is preserved, but the effective number of independent observations within a fold is smaller than the raw window count. The covariate-shift audit (Section 8.3.0.1) and the bootstrap confidence intervals on the 5-fold means therefore use the per-fold mean as the unit of analysis, which sidesteps the within-fold non-independence.

5.4. Baselines and Ablation Designs

We compare G-Code Decoder against the following ablations and baselines, each trained under identical preprocessing, splits, and hyperparameter configuration:

1. **No-shortcuts baseline** (Section 6.2). The headline configuration with positional metadata not exposed to the decoder.
2. **With-shortcuts ablation** (Section 6.10.1). The same model with `window_index`, `total_windows`, and `source_file` hash exposed as additional decoder inputs. Tests whether positional metadata inflates accuracy beyond what sensor signal alone can recover.
3. **Full-window target ablation** (Section 6.10.3). The same encoder but with multi-line per-window targets instead of per-row targets. Tests whether autoregressive row-to-row context within a window resolves the per-row ambiguity discussed in Section 3.3.
4. **Sensor-modality ablation** (Section 6.11). For each of the seven sensor groups, we zero the corresponding channels at the encoder input and evaluate the resulting decoder, across five folds per modality (audio-RMS at $n = 4$; one fold's ablation run was unrecoverable).
5. **Sequence-classifier (pattern-aware) ablation** (Supplementary Section S8). A sequence-classifier head trained jointly with the legacy-token head predicts which whole G-code line the model is reconstructing and biases the token distribution toward that line's tokens.
6. **Vocabulary-precision ablation** (Supplementary Section S8). A coarser 2-digit numeric bucketing reduces vocabulary size by approximately $5\times$.
7. **Noise-augmentation ablation** (Section 6.13). Gaussian noise injection on sensor channels, feature dropout, and token masking during training [28,29,59].
8. **Leave-one-class-out generalization** (Section 6.12). For each of the nine operation classes, train without that class in the training partition and evaluate on its held-out test samples.

5.5. Statistical Analysis

For each five-fold ablation, we report the mean and standard deviation across folds, one-way analysis of variance (ANOVA) against the no-shortcuts baseline, and Cohen's d effect size. ANOVA is computed with SciPy [64]. Bootstrap 95% confidence intervals use 10,000 percentile resamples with the NumPy default RNG seeded at 42, matched to the calibration-split convention of Section 6.7. For paired comparisons we additionally report the Hedges' small-sample-corrected effect size $g_z = J \cdot d_z$ with $J = 1 - 3/(4(n-1)-1)$. At $n = 5$ folds $J \approx 0.80$, so g_z is uniformly $\sim$20% smaller than d_z throughout this paper. Significance levels are reported as $*\,p < 0.05$, $**\,p < 0.01$, $***\,p < 0.001$. Reported $\pm$ values follow each upstream audit script's native convention: NumPy default population s.d. for the headline and sensor-ablation tables, and ddof=1 sample s.d. for the K-spectrum and no-numeric tables. The typical $\sim$10–15% spread between the two conventions at $n = 5$ does not change any significance call or deployment recommendation.

Inferential scope and a caution on $n = 5$.

The unit of analysis is the per-fold mean, so every parametric test reported here is computed on $n = 5$ observations per group (the audio-RMS modality-ablation group is $n = 4$, one fold unrecoverable). We therefore treat all p-values as *descriptive* rather than confirmatory. We read the bootstrap 95% intervals as a smoothed (not coverage-calibrated) indication of cross-fold spread. We take the nonparametric tests (Mann–Whitney U, Wilcoxon) as the primary inferential statement. The conclusions rest on effect sizes large relative to the cross-fold spread rather than on threshold-crossing p-values. Single-fold pilots are labeled as such and excluded from every quantitative or deployment claim. The full caution is in Supplementary Materials Section S38.

5.6. Software and Compute

The decoder is implemented in PyTorch and trained on NVIDIA RTX A6000 GPUs. Preprocessing and analysis use scikit-learn [55] and SciPy [64]. The scikit-learn `HistGradientBoosting` non-neural baseline (Section 6.1) is used in place of XGBoost [65] because the available XGBoost build was CUDA-only and failed on the CPU inference host (Supplementary Section S19.8). Hyperparameter sweeps were tracked with Weights and Biases. Full code and configuration files are released with the manuscript.

6. Results

We address the four research questions stated in Section 1. Section 6.2 establishes the RQ1 headline; Sections 6.3–6.10.2 unpack it along the categorical-versus-numeric axis; Sections 6.10, 6.11, and 6.12 address RQ2–RQ4; Supplementary Section S8 and Section 6.13 report supplementary cross-cuts.

6.1. Non-Neural Baseline Comparison

Two lightweight classifiers, a scikit-learn `HistGradientBoosting` (HGB) tree-booster and the two-layer MLP probe of Section 4.5, operate on the same mean-pooled encoder embedding the decoder consumes. They anchor the headline by isolating the contribution of the decoder's autoregressive multi-head architecture from the encoder embedding itself. On the binary fields, both hit the always-most-common-class data-coverage ceiling ($\approx$ 0.88 *has-X*, 0.99 *has-F*); these ceilings are the correct reference points for the decoder's per-field numbers. On multi-class command identity, both fall well short (HGB macro-F1 0.23, MLP $\approx$ 0.24), consistent with the per-row modal-row ceiling (Section 4.5). On *has-Z*, the one genuinely within-window-diverse field, HGB reaches macro-F1 0.86, confirming that the pooled embedding carries some per-row Z signal. The full per-task baseline table and analysis are in Supplementary Materials Section S23.

6.2. RQ1: Headline Accuracy and Per-Field Recoverability

Table 1 reports the headline accuracies and macro precision/recall/F1 for each prediction head across five folds, under both teacher-forced and autoregressive evaluation. The categorical heads reach 0.9838 ± 0.0085 type accuracy, 0.8875 ± 0.0558 command accuracy, 0.9931 ± 0.0036 parameter-type accuracy, and 0.9888 ± 0.0063 pooled motion-sign accuracy. These accuracies are identical under both regimes because the heads are per-position classifiers on the encoder memory and do not consume the autoregressive token stream. The autoregressive-token heads diverge between regimes: token accuracy is 0.7811 ± 0.0216 (TF) vs. 0.2148 ± 0.0685 (AR), and numeric accuracy is 0.5850 ± 0.0331 (TF) vs. 0.1038 ± 0.0322 (AR). Sequence exact-match is reported only under AR (per-fold $\{0.015, 0.000, 0.028, 0.056, 0.032\}$, mean 0.0263 ± 0.0185; `audit/ar_aggregate.json`). A sequence-level TF score is not separately well-defined: the metric requires generating a full output, and under teacher forcing the model is conditioned on the true prefix at every position, so the resulting "sequence" score is the per-token TF accuracy collapsed to an all-or-nothing test. The AR sequence number of 0.0263 is therefore the deployment-relevant figure. Section 7.3 unpacks the mode-collapse mechanism behind the AR drop; the evidence for partial rather than total collapse is the output-diversity audit reported there. The fold-to-fold spread per metric is visualized in Supplementary Materials Section S39. The per-class precision, recall, and F1 for the command and parameter-type heads are charted in Supplementary Materials Section S24 (Figure S32).

The two regimes factorize along the head boundary that Section 3.4 formalizes. TF/AR invariance is a structural property of the encoder-memory classifiers; the TF–AR gap on the token and numeric heads is the exposure-bias signature of autoregressive coupling. The two regimes therefore characterize different things, and the per-field recoverability analysis in the sections that follow reads them together. We reserve the word *recovery* for the per-field categorical heads (command, parameter-type, sign), which are sensor-conditioned and TF/AR-invariant. End-to-end generative reconstruction is at floor: autoregressive whole-sequence exact-match is 0.0263 ± 0.0185, i.e. fewer than 3% of windows are reproduced verbatim. This paper therefore makes no claim of generative G-code reconstruction; the contribution is per-field categorical recoverability, not toolpath reconstruction.

Robustness to the numeric-target patch.

The numeric-digit target pipeline contained a float-epsilon artifact at the finest decimal slots, corrected by a round-encoding retrain (`checkpoints/full_window_5fold_patched/`). The headline and categorical metrics are *robust* to the patch: patched vs. released 5-fold TF values are command 0.891/0.888, token 0.774/0.781, numeric 0.570/0.585, type 0.983/0.984, param-type 0.993/0.993, and AR sequence 0.029/0.026, so every pooled/headline metric drifts ≤ 1.5 pp. We therefore report Table 1 on the released checkpoint. The patched retrain is used only for the numeric tier (the per-axis MAE and the per-digit bathtub of Section 6.4), where the correction is concentrated: it moves the finest digit slot (slot-5) accuracy from 0.77 to 0.99 while leaving the pooled numeric metric near-unchanged. The supervisory `target_tokens` are identical between runs; the corpus-weighted target-correction rate is 32.5% of numeric positions (30.1% terminal-zero $9 \rightarrow 0$ fixes).

Three caveats govern how these numbers may be read. **First, they are closed-vocabulary, in-distribution** (coverage-repair regime, Section 5.3; 97.6%/99.0% verbatim test-line overlap), so the command accuracy of 0.8875 is a within-vocabulary classification result and *not* a deployment-ready open-vocabulary number. Second, the frozen encoder was trained on the same file-level fold partitions, so the headline 0.888 contains an unmeasured representation-reuse component on top of the per-instruction sensor recovery this paper claims (see Supplementary Materials Section S21). Third, three companion numbers anchor the headline: (i) the leave-one-class-out (LOCO) cross-class command accuracy of 0.213 ± 0.191, the open-vocabulary floor (Section 6.12); (ii) the class-prior control (Section 7.1.0.2), the right home for the within-class sensor-driven increment over the operation-class modal predictor; and (iii) the AR collapse described above.

The command head's macro F1 (0.879 ± 0.108) is moderately lower than its raw accuracy (0.8875 ± 0.0558), reflecting the long-tailed class distribution: G1 carries $\sim$830 test instances per fold, G3 $\sim$315, G0 $\sim$65, and G53 and M30 $\sim$6 each. Recovery is strong across all six classes (per-class F1 in Table 2); no class collapses. The lowest per-class F1 is G2 at 0.82. The confusion matrix (Supplementary Materials Section S24, Figure S33) attributes the G2 result to bidirectional G1/G2 confusion in arc-rich windows: the largest off-diagonal cells, G1→G2 ($n = 373$) and G2→G1 ($n = 253$), are directionally symmetric, indicating local-pattern ambiguity between linear cuts and arcs sharing the same axis signature rather than a one-way bias. Per-class minimum detectable effects from the power analysis ($\sim$2.3 pp on G1, $\sim$8 pp on G2/G3, $\sim$19 pp on G0 at $\alpha = 0.05$, $\beta = 0.20$; `audit/extended_rigor_audits.json`, Section 8.3) bound the resolution of the per-class column; differences below these sizes are not statistically distinguishable from sampling noise.

Table 1. Headline test-set metrics for the G-Code Decoder under 5-fold file-level cross-validation, full-window targets, no positional metadata exposed. Mean ± std over 5 folds. Two evaluation regimes are reported: teacher-forced (TF), where each step receives the ground-truth previous token, and autoregressive (AR, greedy / beam-width 1), where the decoder consumes its own previous predictions. The categorical heads — type, command, parameter-type, sign — report identical accuracies under both regimes because they are independent per-position classifiers on the encoder memory; only the autoregressive-token heads (token, sequence, numeric) diverge.

Head	TF Accuracy	AR Accuracy	Macro Precision	Macro F1
Token	0.7811 ± 0.0216	0.2148 ± 0.0685	0.4099 ± 0.0406	0.3899 ± 0.0405
Sequence‡	–	0.0263 ± 0.0185	–	–
Type	0.9838 ± 0.0085	0.9838 ± 0.0085	0.9091 ± 0.0352	0.8833 ± 0.0451
Command	0.8875 ± 0.0558	0.8875 ± 0.0558	0.9342 ± 0.0407	0.8790 ± 0.1084
Param-type	0.9931 ± 0.0036	0.9931 ± 0.0036	0.9914 ± 0.0073	0.9857 ± 0.0130
Sign	0.9888 ± 0.0063	0.9888 ± 0.0063	0.9658 ± 0.0215	0.9724 ± 0.0157
Numeric (overall digit)	0.5850 ± 0.0331	0.1038 ± 0.0322	–	–

Macro precision/recall/F1 are unweighted means over all observed classes; values are stable across TF and AR for the categorical heads, so a single column is reported. Numeric is the digit-head accuracy pooled across all six digit positions; the per-position bathtub is shown in Supplementary Materials Section S39 (discussed in Section 6.4). The TF / AR gap on the token and numeric heads is the exposure-bias signature analyzed in Section 7.3: the decoder's categorical signal is robust to autoregressive deployment, while its full-sequence token reproduction is not. ‡ Sequence exact-match is reported only under AR; a sequence-level TF score is not separately well-defined (see Section 6.2).

Table 2. Per-class precision / recall / F1 for the motion-command head, 5-fold mean ± std. Support is the approximate number of test instances per fold; per-class confidence intervals widen for the smallest-support classes, but no class collapses in 5-fold-mean F1 (the smallest-support classes G53 and M30, $n \approx 6$/fold, nonetheless show wide per-fold recall variance and are underpowered).

Class	Precision	Recall	F1	Support
G0	0.946 ± 0.052	0.890 ± 0.142	0.917 ± 0.079	$\sim$65
G1	0.929 ± 0.024	0.886 ± 0.097	0.905 ± 0.049	$\sim$830
G2	0.811 ± 0.119	0.851 ± 0.039	0.823 ± 0.067	$\sim$400
G3	0.927 ± 0.072	0.913 ± 0.063	0.917 ± 0.043	$\sim$315
G53	1.000 ± 0.000	0.846 ± 0.327	0.876 ± 0.206	$\sim$6
M30	0.950 ± 0.112	0.769 ± 0.213	0.836 ± 0.150	$\sim$6

The parameter-type head shows a different pattern: macro F1 0.986 ± 0.013 closely tracks accuracy because the four well-supported axis letters (X, Y, Z, R) are each predicted at F1 ≥ 0.989. The thin-support F axis reaches F1 0.958 ± 0.042 on only $\sim$40 test instances; the per-class breakdown table is in Supplementary Materials Section S39.

Supplementary Materials Section S24 (Figures S32 and S33) reproduces the per-class precision/recall/F1 bar charts and row-normalized confusion matrices for the command, parameter-type, and type heads: command F1 spans 0.82–0.92 across the six classes, off-diagonal mass concentrates on the G2/G3 pair, and parameter-type recovery is near-perfect on X/Y/Z/R. The motion-sign and per-digit-value confusion matrices (Supplementary Section S13.4, Figure S19) visually corroborate the per-axis sign accuracies of Table 4 and the diagonal-plus-adjacent-band structure that follows from the per-digit entropy decomposition (Section 6.4).

6.3. Per-Axis Recoverability

For each axis letter we extract three quantities from the decoded G-code: (i) *has-axis*, whether the axis appears in the predicted block; (ii) *sign*, the sign of the motion when the axis is present (POSITIVE, NEGATIVE, ABSENT); and (iii) *value MAE*, the regression mean absolute error of the numeric value, computed only over samples where both the predicted and true blocks carry the axis. Table 4 reports these per axis. Table 3 provides the per-field frequency context needed to interpret Table 4: fields with near-zero presence cannot be meaningfully evaluated regardless of model accuracy.

Table 3. Per-field positive-support frequency in the V8 corpus. Accuracy numbers for a field with near-zero support (e.g. F, S, I, J in per_row mode) are dominated by the always-absent baseline and cannot be interpreted as evidence of sensor-driven recoverability. Source: direct count over the V8 preprocessed NPZ files, fold 1.

Field	per_row train	per_row test	full_window train	full_window test
X	90.4%	88.5%	100.0%	100.0%
Y	88.6%	86.6%	100.0%	100.0%
Z	35.8%	31.4%	29.7%	32.6%
F	**0.4%**	**0.6%**	18.8%	22.0%
S	**0.0%**	**0.0%**	**0.0%**	**0.0%**
R	15.2%	17.2%	88.1%	88.6%
I	**0.0%**	**0.0%**	**0.0%**	**0.0%**
J	**0.0%**	**0.0%**	**0.0%**	**0.0%**

Bold cells indicate fields whose accuracy in Table 4 cannot be meaningfully evaluated due to insufficient positive support; recovery of these fields would require a corpus with non-trivial F/S/I/J coverage, beyond the present data.

Table 4. Per-axis G-code recoverability across 5 folds (mean ± std). **Value-MAE column re-derived on the float-epsilon-corrected retrain** (`checkpoints/full_window_5fold_patched/`; `audit/patched_numeric_analysis.json`); the categorical (has-axis, sign) columns are checkpoint-invariant. *has-axis Acc / F1* are the classification metrics for whether the axis is present in the decoded text; *sign* is the direction of motion (POSITIVE / NEGATIVE / ABSENT); *value MAE* is the mean absolute error of the numeric value, computed only on samples where both the predicted and true G-code carry the axis. The headline recoverability claim is restricted to the four well-supported axes **X, Y, Z, R**. The F row (†) is reported *for the full-window regime only*, where F appears in 22% of windows; in the per-row regime F support is only 0.6% and is not evaluable (Table 3). I, J, S have effectively zero positive support in both regimes and are not evaluable. The per-axis sign-accuracy asymmetry (Y 0.997 vs. X 0.942 vs. Z 0.938) is *prior-driven*: Z is the only sign-bearing axis whose sign carries non-trivial classification load ($H_Z^{\text{sign}} = 1.04$ bits), whereas Y's sign is prior-deterministic ($H_Y^{\text{sign}} = 0.08$ bits) and X's is near-deterministic ($H_X^{\text{sign}} = 0.38$ bits); the per-axis sign source-entropy decomposition is developed in the companion limit paper [33], so the high Y/X sign accuracy reflects the prior, not sensor discrimination.

Axis	has-axis Acc	has-axis F1	Sign Acc	Value MAE	Presence Recall
X	1.000 ± 0.000	1.000 ± 0.000	0.942 ± 0.033	0.248 ± 0.140	1.000 ± 0.000
Y	1.000 ± 0.000	1.000 ± 0.000	0.997 ± 0.006	0.205 ± 0.080	1.000 ± 0.000
Z	1.000 ± 0.000	1.000 ± 0.000	0.938 ± 0.023	0.009 ± 0.006	1.000 ± 0.000
R	0.994 ± 0.012	0.983 ± 0.034	1.000 ± 0.000	0.021 ± 0.011	1.000 ± 0.000
F†	0.960 ± 0.023	0.936 ± 0.038	1.000 ± 0.000	1.115 ± 0.350 (inch/min)	0.851 ± 0.093
S, I, J	*insufficient support; see Table 3*	—	—	—	—

X, Y, Z, R presence is fully recovered. The sign asymmetry is prior-driven, not a sensor-separability ranking: Y (0.997) and X (0.942) sign accuracy reflect their near-deterministic sign priors ($H_Y^{sign} = 0.08$, $H_X^{sign} = 0.38$ bits), while Z (0.938) is the only axis whose sign is genuinely balanced ($H_Z^{sign} = 1.04$ bits) and therefore the only one carrying real sensor-driven sign signal. Value MAE is small for Z (0.009) and R (0.021) and larger for X/Y (0.21–0.25), all in inches *(the corpus's native unit; the platform runs in inch mode). Against the per-axis coordinate span over the corpus (X ≈ 3.8, Y ≈ 3.8, Z ≈ 0.7, R ≈ 7.9 in) these MAEs are ≈ 6.5% and 5.4% of span on the large planar X/Y axes but only ≈ 1.2% and 0.3% on the shallow-depth Z and arc-radius R axes — coordinate value is recovered to coarse resolution on X/Y and near-exactly on Z/R. These are the patched-retrain values (Z is 0.009 patched vs. 0.012 on the released checkpoint recomputed identically; an earlier reported 0.042 was a pre-patch analysis-pipeline artifact, not a checkpoint difference). The F-row MAE is on a different scale because feed-rate values themselves are larger (four distinct posted values, 7.3–40 in/min, in this corpus).*

Four patterns emerge. First, axis-presence detection is uniformly high: X, Y, and Z reach perfect 5-fold has-axis accuracy (1.000 ± 0.000), R reaches 0.994 ± 0.012, and F reaches 0.960 ± 0.023 on its thin support. Second, motion-sign recovery is axis-asymmetric, but the asymmetry is *prior-driven*, not a sensor-separability ranking. Y-sign (0.997 ± 0.006) and X-sign (0.942 ± 0.033) ride near-deterministic sign priors ($H_Y^{\text{sign}} = 0.08$, $H_X^{\text{sign}} = 0.38$ bits). Z-sign (0.938 ± 0.023) is the only sensor-bearing sign axis and the only one whose sign distribution is genuinely balanced ($H_Z^{\text{sign}} = 1.04$ bits); Z's lower accuracy therefore reflects a harder sensor-driven binary classification, not weaker sensor coupling. Third, the patched-retrain numeric value MAE (`audit/patched_numeric_analysis.json`) is small for Z (0.009 patched; 0.012 on the released checkpoint recomputed identically) and arc-radius R (0.021) but larger for X (0.248) and Y (0.205), all in inches. This ordering is consistent with the per-digit decomposition (Section 6.4) localizing the loss to middle digit positions; the wide X fold spread (± 0.14) reflects variation in large X-axis moves. Fourth, spindle (S) and arc parameters (I, J) have effectively zero positive support (Table 3) and cannot be meaningfully evaluated. The per-axis summary chart (has-axis F1, sign accuracy, and value MAE side by side) is in Supplementary Materials Section S39.

The per-axis distribution of $|\hat{v} - v|$ that the value-MAE column averages over (Supplementary Materials Section S24, Figure S34) is bimodal. X/Y/Z errors are heavy-tailed, with substantial mass at 10^{-2} and 10^{-1}-inch magnitudes; R errors are tightly concentrated near exact-match (the discrete arc-radius vocabulary). The per-axis MAE summary in Table 4 is therefore a midpoint between two structurally different regimes within each axis, not a unimodal noise floor.

6.4. Numeric Digit Head Decomposition

The digit head achieves 0.5850 ± 0.0331 mean accuracy under teacher-forced evaluation, dropping to 0.1038 ± 0.0322 under autoregressive evaluation (Table 1). The per-position decomposition below is computed at the TF condition on the float-epsilon-corrected retrain (`audit/patched_numeric_analysis.json`), so the entropy and accuracy axes are consistent at the finest digits. The pooled 5-fold per-slot accuracy is bathtub-shaped (curve in Supplementary Materials Section S39). Slot 0 (10^1, tens) reaches 0.9999; slot 1 (10^0, units) 0.9223 ± 0.0180; slot 2 (tenths) 0.7478 ± 0.0253; slot 3 (hundredths) 0.5465 ± 0.0269. Slot 4 (10^{-3}, thousandths) bottoms out at 0.4165 ± 0.0239; this is the *true precision floor*, since the thousandths is the finest digit the CAM postprocessor emits. Slot 5 (10^{-4}, ten-thousandths) returns to 0.9931 ± 0.0067 because the V8 tokenizer quantizes X/Y/Z onto a 0.001 grid ($p_X = p_Y = p_Z = 10^{-3}$; Supplementary Materials Section S22), making the ten-thousandths digit of the tokenized target structurally near-zero entropy (0.31 bits) and therefore deterministically reproduced rather than information-bearing. The slot-5 "upturn" is a tokenizer representation choice, not a source or sensor limit; the companion limit paper [33] shows that R, quantized on a 0.0001 grid, retains its ten-thousandths digit.

The decoder therefore behaves as a *coarse-resolution coordinate recoverer*: it reproduces the physically- or geometrically-constrained high-order digit positions and the deterministic trailing digit, and it is bounded at a near-uniform-noise ceiling at the thousandths place, where the CAM postprocessor's interpolation residuals dominate. The patched correction strengthens the entropy–accuracy anticorrelation; it makes the slot-5 "upturn" an honest low-entropy (deterministic) structure rather than the float-epsilon encoding artifact it appeared to be before the retrain.

The per-(axis, slot) source entropy of the toolpath corpus explains the shape of this profile: the digit head tracks an information-theoretic ceiling that is near-uniform ($\approx \log_2 10$ bits) at the thousandths and low on the physically constrained high-order places and the deterministic trailing digit. The companion limit paper [33] characterizes this ceiling (corrected pooled $n = 6$ Pearson -0.89; per-(axis,slot) $n = 30$ Pearson -0.83).

Per-axis sign recovery (Z is the only sensor-bearing sign axis).

The motion-sign head's 0.9888 headline accuracy (Table 1) is dominated by prior-deterministic axes and should not be read as a sensor-recovery figure. Z is the only axis whose sign carries non-trivial classification load ($H_Z^{\text{sign}} = 1.04$ bits): its coordinates split roughly evenly between positive rapids above the stock and negative cuts into the stock. Y motion is $\sim 99\%$ positive ($H_Y^{\text{sign}} = 0.08$ bits) and X $\sim 93\%$ positive ($H_X^{\text{sign}} = 0.38$ bits), so the head's high accuracy on those axes is the sign prior re-expressed, not sensor evidence, and it would fail on a workpiece of different orientation or origin. The deployment-relevant per-axis sign accuracies are therefore the X/Y/Z numbers in Table 4 (0.94/1.00/0.94 5-fold means), with Z carrying essentially all the discriminative work. The companion limit paper [33] quantifies this asymmetry through the per-axis sign-entropy decomposition.

6.5. Per-Operation-Class Recoverability

Autoregressive predictions stratify sharply by operation class, and the stratification has a single mechanistic cause rather than a per-class capability gradient. One class, `face` ($n = 144$ test windows), reaches token 0.56 / numeric 0.40 against a pooled AR baseline of 0.215/0.104, while every other class falls into a narrow 0.09–0.25 token / 0.01–0.15 numeric band. The most-frequent non-facing class, `pocket150025` ($n = 124$), sits at 0.09/0.01, floor-level despite ample test support. This is the per-class signature of the mode-collapse mechanism analysed in Section 7.3: the corpus-modal AR output the decoder converges to *is* a facing-pattern toolpath. `face` windows therefore score by coincidental agreement with the modal trajectory rather than by genuine sensor-driven discrimination, and any class with a different toolpath signature scores near floor regardless of test sample count. The headline categorical recoverability (Section 6.2) remains the central per-class claim; the per-class *numeric* story is constant-mode-collapse with one outlier and should not be read as "`face` is easier to decode." The full per-fold breakdown, methodology, and per-class numbers are in Supplementary Materials Section S1.

6.6. Test-Time Sensor-Noise Sensitivity

Re-evaluating fold 1 of the baseline checkpoint while injecting additive Gaussian noise into the frozen-encoder memory shows the categorical heads are essentially flat across $\sigma \in [0, 2]$ (up to twice the signal magnitude): command-head accuracy stays in 0.78–0.82, type at 0.97, and parameter-type at 0.98. This experiment probes the encoder-to-decoder interface (robustness to conditioning-vector perturbation), not raw-sensor degradation. The deployment-relevant reading is that the categorical-recovery story holds under encoder-memory perturbation, the relevant failure mode for a frozen-encoder deployment validated upstream. Full per-σ results, the figure, and the sign-head audit-script caveat (which does *not* affect the headline sign accuracy) are in Supplementary Materials Section S10.1.

6.7. Calibration of the Categorical Heads

A high-accuracy classifier may still be poorly calibrated. The type and parameter-type heads are well-calibrated out of the box (fold-1 ECE 0.015 and 0.020 at 10 bins). The command head is moderately *overconfident* (fold-1 ECE 0.122, MCE 0.207 at 0.780 accuracy against 0.902 mean confidence), consistent with the LOCO collapse signature (Section 6.12): it places high mass on G1 even when the true command is not G1. Post-hoc temperature scaling (a single learned T per head, fit by NLL on a held-out calibration half of each fold) reduces command-head cross-fold ECE from 0.050 ± 0.038 to 0.020 ± 0.011 without changing accuracy ($T_{\text{cmd}} = 1.34 \pm 0.25$ does not move the argmax). We recommend it as the default deployment-time calibration step; the Brier/Murphy decomposition confirms it targets the reliability term and preserves discrimination. The full ECE/MCE breakdown, Brier/Murphy decomposition, per-fold fitted temperatures, reliability diagram, and the parameter-type worst-bin MCE trade-off caveat are in Supplementary Materials Section S2.

6.8. Output-Position Failure Distribution

A token-position breakdown isolates a single concentrated failure rather than a distributed error budget. In the per-row formulation, position 1 (the first address letter following the command token) is recovered at only 0.240, a sharp drop from 0.761 at position 0 and 0.743–0.929 at positions 3–5. This drop is the signature of within-window row-identification ambiguity. The full-window formulation supplies the previous row's output as autoregressive context and lifts position 1 to 0.622 ± 0.027; the $+38$ pp gain on position-1 token recovery localizes the full-window win (Section 6.10.3) to row-disambiguation. Past the row boundary both formulations recover monotonically (0.900–0.940 through position 100), so the position-1 failure is structural ambiguity, not length degradation. The full per-position table and curve are in Supplementary Materials Section S10.5.

6.9. Failure-Mode Analysis

Bucketing the 20 worst-edit-distance sequences per fold (100 across 5 folds) by the structural difference between predicted and target token streams gives: dropped command 17, wrong command 1, hallucinated command 8, value-only error 37, and "other" 37 (same-command structural mismatches dominated by Y↔X address-letter swaps). Command-identity failures together account for 26% of the worst sequences. The worst failures are predominantly structural rather than arithmetic; Section 7.1.0.1 interprets this. Representative examples per bucket are in Supplementary Section S12.

6.10. RQ2: Metadata vs. Sensor and Per-Row vs. Full-Window

RQ2 asks two related questions: (i) does positional metadata inflate accuracy beyond what sensor data alone provides? and (ii) does the per-row target formulation introduce a row-identification ambiguity that full-window targets resolve? Table 5 provides the headline numbers; Sections 6.10.1 and 6.10.3 unpack the two effects.

Table 5. Ablation matrix. Each ablation row reports both teacher-forced (TF) and autoregressive (AR, beam-width-1 greedy) test accuracy where the metric depends on the regime. The categorical heads (type, command, param-type, sign) are independent classifiers on the encoder memory and report identical accuracies under both regimes; a single value is given. The pattern-aware row is a fold-1 pilot (Supplementary Section S8); the sensor-modality effect is summarized here and broken out per modality in Table 7 (5-fold; audio-RMS $n = 4$). Numbers from `outputs/decoder20260511/audit/ar_aggregate.json` (5-fold AR) and the per-fold `metrics.json` (5-fold TF), commit `db246b0`.

Configuration	Token (TF / AR)	Sequence (AR)	Type	Command	Param-type	Numeric (TF / AR)
Baseline (no shortcuts, full-window)	0.781 ± 0.022 / 0.215 ± 0.069	0.026 ± 0.019	0.984 ± 0.009	0.888 ± 0.056	0.993 ± 0.004	0.585 ± 0.033 / 0.104 ± 0.032
+ positional metadata	0.754 ± 0.033 / $\mathbf{0.290 \pm 0.016}$	$\mathbf{0.102 \pm 0.022}$	0.982 ± 0.011	0.882 ± 0.056	0.993 ± 0.004	0.532 ± 0.054 / $\mathbf{0.184 \pm 0.017}$
+ sequence classifier (pattern-aware, fold-1)	0.694 / –	0.010	0.975	0.536	0.952	0.392 / –
− any single sensor modality (5-fold; rms $n = 4$)	command 0.449–0.484 (uniform −40 to −44 pp; between-modality ANOVA $p = 0.998$); token ∼0.74; type / param-type within ∼1–4 pp of baseline. Full per-modality breakdown: Table 7.					

The TF / AR gap is the exposure-bias signature analyzed in Section 7.3: token and numeric heads consume their own previous predictions under AR and accumulate compounding errors over the ∼1,000-token full-window sequence. Categorical heads are unaffected. The + positional metadata *ablation reverses sign across regimes: under TF it is mildly harmful or neutral, under AR it produces +7.5 pp on token, +8.0 pp on numeric, and +7.6 pp on sequence, with ∼4× tighter fold-to-fold variance on the token head. Section 6.10.1 interprets this as mitigation of autoregressive mode collapse rather than a positional shortcut.*

6.10.1. Positional-Metadata Ablation

Re-training the decoder *with* positional metadata exposed (window index, total-window count, source-file hash) produces the per-metric effect sizes in Table 5 (the + *positional metadata* row). One-way ANOVA across the 5 folds of each configuration shows a clean factorization:

- Categorical heads are unaffected by positional metadata: negligible effect sizes, $\Delta \approx 0$ on command/type/param-type/sign, $F < 0.05$ under both AR and TF. At $n = 5$ folds per group these are descriptive null effects, not confirmed equivalences.
- Autoregressive heads shift by token $\Delta = +7.5$ pp ($F = 4.5$), sequence $\Delta = +7.6$ pp ($F = 27$), and numeric $\Delta = +8.0$ pp ($F = 19$). The effect sizes and the consistent positive sign across folds carry the conclusion, not the p-values: with $n = 5$ folds per group every p-value here is descriptive, so we report each to one significant figure ($p \sim 10^{-3}$ for the sequence and numeric heads, $p \sim 0.07$ for the borderline token head) and rest the claim on the $+7.5$–8.0 pp positive increments. Under TF evaluation the token and numeric heads show no positive increment (both small and non-positive in 5-fold means).

Categorical heads (command, type, parameter-type, sign, per-axis presence and sign) are recoverable from sensors alone, and exposing positional metadata does not improve them under any evaluation regime. The with-shortcuts 5-fold command-accuracy mean is 0.882 ± 0.056 (Table 5), statistically indistinguishable from the no-shortcuts 0.8875 ± 0.0558 baseline. These heads are independent per-position classifiers on the encoder memory; their accuracies are identical under teacher-forced and autoregressive decoding (Section 3.4).

Autoregressive-token heads (token, numeric, sequence) depend on the evaluation regime. Under teacher-forced evaluation the positional metadata provides no lift and is mildly harmful (-2.7 pp on token, -5.3 pp on numeric in 5-fold means). Under autoregressive deployment-true evaluation the picture reverses: positional metadata produces approximately $+7.5$ pp on token and $+8.0$ pp on numeric accuracy (Section 7.3). The mechanism is mitigation of autoregressive mode collapse. In the no-shortcuts AR setting the decoder converges to a corpus-modal token sequence regardless of sensor input; exposing the window index gives the decoder a per-window anchor that breaks the modal trajectory on a fraction of test samples. Positional metadata is therefore not a "shortcut" that inflates sweep-time accuracy but a deployment-time disambiguation signal that partially compensates for exposure bias in long-sequence autoregressive decoding. Bootstrap percentile cross-fold spread intervals (95%, not coverage-calibrated at $n = 5$) for both configurations appear in Supplementary Materials Section S39.

Decomposing the collapse mechanism.

The position-metadata mitigation establishes that per-window anchoring partly breaks the modal trajectory. A complementary no-numeric retraining ablation decomposes what the collapse is made of, isolating a measurable numeric-token-desynchronization contribution from a dominant intrinsic mode-collapse of the structural token stream itself; the latter, not the numeric vocabulary, is the deployment-relevant bottleneck (full statistics in Section 6.10.2).

Multiple-comparisons correction on the drilled grid.

The per-class / per-axis / per-position ANOVA grid (methodology in Supplementary Section S19.7) covers 8 ablation configurations (noise augmentation and the seven zeroed-modality ablations) against the no-shortcuts baseline, drilled across the per-class precision/recall/F1 for type, command, parameter-type, sign, and the per-digit-value / per-position numeric heads, for a total of 840 (ablation, drilled-metric) tests (`audit/anova_per_class_results.json`). Benjamini–Hochberg false-discovery-rate (BH-FDR) adjustment ($\alpha = 0.05$) flags 457/840; the more conservative Holm-Bonferroni adjustment flags 167/840. We read this as a *robustness sweep*, not as 457 established effects: each test is an $n = 5$ ANOVA, so FDR/Holm control the decision rule's error rate but cannot supply the power a five-point test lacks. A few large-effect ablations carry the signal and need no multiplicity machinery: the noise-augmentation ablation (per-class command F1 drops of 0.3–0.7 for G0/G1/G2/G3) and the uniform command-head degradation shared by all seven modality ablations (consistent with the non-significant between-modality ANOVA in Section 6.11). Per-class cells with $n_{\text{support}} \leq 50$ are underpowered regardless of adjustment (Section 8.3). The correct reading of the baseline-vs-ablation grid (Supplementary Materials Section S39) is therefore the effect size and the sign of Δ, not the threshold-crossing of any adjusted p-value.

6.10.2. No-Numeric Retraining: Decomposing the Autoregressive Collapse

Roughly half of every G-code line's positions are coordinate *values* (Section 6.4), each a hard selection from the ~2,400-entry numeric vocabulary. A single mispredicted value desynchronizes the autoregressive decode of the remainder of the line. To test whether the AR collapse of Section 7.3 is *driven* by the numeric tokens, we retrain the decoder with every coordinate value collapsed to a single placeholder token (24-entry structural vocabulary), so that a numeric error can never desynchronize the decode. The frozen encoder, 5-fold split, and decoder hyperparameters are otherwise unchanged (full setup in Supplementary Materials Section S29).

Table 6 reports the comparison. **Under teacher forcing** the no-numeric decoder reaches structural token accuracy 0.927 ± 0.024 against the headline decoder's 0.960 ± 0.020, and structural sequence exact-match 0.068 ± 0.017 against the headline's 0.213 ± 0.048; collapsing the varied numeric tokens to a single placeholder therefore carries a real but modest cost. **Under autoregressive decoding, the two effects decompose.** Structural per-token accuracy improves by +8.8 pp (0.405 ± 0.143 vs. 0.317 ± 0.116; 5/5 folds positive, paired $t = 3.56$, $p = 0.024$, Wilcoxon $p = 0.06$). Structural sequence exact-match is essentially unchanged (0.066 ± 0.014 vs. 0.058 ± 0.039, 2/5 folds positive), so the deployment-relevant window-level recovery does not improve. The mode-collapse mechanism of Section 7.3 therefore decomposes into (a) **numeric-token desynchronization**, eliminated by the ablation, and (b) a dominant **intrinsic autoregressive mode-collapse of the structural token stream itself**, which the ablation does not affect. A matched-methodology control (against the $K = 2{,}418$ T3.1 cell at the same schedule) shows the numeric-vocabulary collapse contributes no additional lift beyond the training-schedule change T3.1 already absorbs; this downgrades the standalone "numeric-desynchronization contributes ~ 9 pp" framing while strengthening the dominant-intrinsic-collapse reading. The attribution caveat and the full two-contributor decomposition are in Supplementary Materials Section S29.

Table 6. No-numeric retraining ablation: structural-skeleton recovery of the headline decoder versus a decoder retrained with every coordinate value collapsed to a single placeholder token. *Structural* restricts the metric to command-code and axis-letter positions, excluding numeric/placeholder positions. 5-fold mean $\pm$ std, full-window targets, under teacher-forced (TF) and autoregressive (AR) evaluation. Under TF the no-numeric decoder is modestly weaker on structure (-3 pp per-token, -15 pp per-window) — the cost of collapsing the lexical variety of the numeric vocabulary. Under AR the two effects decompose: per-token structural accuracy improves +8.8 pp (paired across folds, 5/5 folds positive, $p = 0.024$, Wilcoxon $p = 0.06$) — the numeric-desynchronization component of the collapse, whose standalone contribution the matched-methodology control of Section 6.10.2 bounds — but per-window structural exact-match is unchanged, identifying intrinsic structural mode-collapse as the dominant window-level mechanism.

	Headline decoder		No-numeric decoder	
Metric	TF	AR	TF	AR
Structural token accuracy	0.960 ± 0.020	0.317 ± 0.116	0.927 ± 0.024	0.405 ± 0.143
Structural sequence exact-match	0.213 ± 0.048	0.058 ± 0.039	0.068 ± 0.017	0.066 ± 0.014

Vocabulary-cardinality spectrum and matched-methodology control (pointer to Supplementary Section S9).

A matched-methodology K-spectrum control ($K \in \{24, 69, 335, 2{,}418\}$ under SS=0/dw=0) shows $K = 2{,}418$ (T3.1) as the directionally strongest variant on every metric (5/5 folds positive on AR struct token, +15.2 pp over the headline; full confound resolution and retraction in Section 8.5). The paired AR-token contrast is Holm $p = 0.071$ (not significant at $n = 5$); the AR-sequence contrast reaches Holm $p = 0.006$. The five-point spectrum, the Friedman/ANOVA omnibus, and the per-pair effect sizes are in Supplementary Section S9.

6.10.3. Target-Mode Ablation (Per-Row vs. Full-Window)

The per-row target formulation (Section 3.3) leaves the decoder without a within-window row-identification signal: every row of a given window shares the same encoder memory, and the decoder must predict a specific row from that shared input. Full-window targeting is the natural alternative. Its supervisory signal is the entire multi-line G-code that fired within the window, generated autoregressively, so row identification comes from the decoder's own output context: row r is predicted conditional on rows $1 \ldots r-1$. The corresponding row in Table 5 reports the resulting accuracies.

The target-mode contrast, per-row (one G-code line per window) versus full-window (the full multi-line sequence within each window), isolates the contribution of autoregressive context to row-level recovery. Holding the architecture, hyperparameters, and training data fixed, command accuracy moves from per-row 0.499 to full-window 0.888, a +38.9 pp gain. Because the command head is an independent classifier on the encoder memory and not autoregressive in its output stream, this lift holds across both teacher-forced and autoregressive evaluation regimes (Section 3.4); it reflects an actual change in what the decoder can recover, not a difference in evaluation condition. Section 7.1.0.1 interprets the lift mechanistically. In per-row mode all rows within a 64-second window share the same encoder memory, so the decoder cannot disambiguate which of ∼60 candidate rows (median 29) it is asked to emit; in full-window mode the multi-line target gives every row a unique within-window position signal, restoring identifiability for the command head.

On the autoregressive-token heads (token, numeric, sequence), the per-row vs. full-window contrast depends on the evaluation regime. In teacher-forced evaluation the full-window formulation reports higher token and numeric accuracy than per-row, but in autoregressive evaluation the difference shrinks substantially because exposure-bias errors compound over the longer full-window sequences. We therefore report the full-window formulation as the deployment-recommended configuration only when categorical recovery is the objective. When end-to-end sequence reproduction is the objective, per-row mode is comparable in autoregressive performance and has the practical advantage of much shorter, parallelizable per-row inference.

6.11. RQ3: Sensor-Modality Ablation

For each of seven sensor modality groups (accelerometer, gyroscope, magnetometer, environmental, color, audio-RMS, electrical) we zero the corresponding sensor channels at the encoder input, then evaluate the decoder. The encoder is not retrained; the zeroing happens only at decoder-inference time, which isolates the marginal contribution of each modality *conditional on the encoder's existing weight allocation*. The result is a *lower bound* on the value of each modality, testing only what the existing encoder extracts from it; the appropriate system-level redundancy test is the encoder retraining of Section 6.11.1. Six of the seven modality groups are evaluated at the full 5-fold protocol ($n = 5$); audio-RMS is evaluated at $n = 4$ because one fold's ablation run was unrecoverable (and was not retrained). The between-modality ANOVA ($F = 0.07$, $p = 0.998$; `audit/sensor_anova_7modality.json`) and the per-modality Welch and Mann–Whitney tests are therefore computed with audio-RMS at $n = 4$ and the other six modalities at $n = 5$. Given the uniform ∼40–44 pp collapse across all seven modalities, the non-significant between-modality verdict does not hinge on the single missing audio-RMS fold.

Uniform collapse, not modality differentiation.

At the inference-time-zeroing test every modality is critical to the command head, and the magnitude is statistically uniform. Zeroing any single modality drops command accuracy from the full-modality baseline of 0.888 ± 0.056 to the 0.449–0.484 band (accelerometer 0.484, environmental 0.474, magnetometer 0.465, gyroscope 0.462, rms 0.460, color 0.457, electrical 0.449; all ±0.05–0.11 across folds), a uniform −40 to −44 pp collapse. A one-way ANOVA across the seven zeroed-modality groups is *not* significant ($F = 0.07$, $p = 0.998$; `audit/sensor_anova_7modality.json`), so the seven modalities are not separable from one another at the present fold count. Every modality *versus* the full-modality baseline *is* significant under a Welch test ($p < 0.001$ in all seven cases, t from −7.1 to −11.5). **No single modality is shown to be more deletable than another at this test:** it confirms joint necessity but cannot rank importance. Because failure-to-reject is not positive evidence for equivalence, we also applied the TOST + BF_{01} machinery of the LOSO study (Supplementary Section S5) to the seven zeroed-modality means ($\Delta \leq 0.035$ across pairs) at a ±0.020 equivalence margin: at $n = 5$ the most likely TOST verdict on several pairs is *inconclusive*, a margin at the resolution limit of a five-block test, consistent with the statement above that this test cannot rank importance. The genuine modality ranking comes from the encoder-retraining LOMO study (Section 6.11.1), not from the inference-time-zeroing test. Token, type, and parameter-type heads are far more robust: token drops ~ 4 pp, type < 1 pp, and parameter-type

$\sim$ 1.5 pp across every ablation. Table 7 reports the per-modality numbers (six modalities at $n = 5$, audio-RMS at $n = 4$); the bar chart is in Supplementary Materials Section S39.

Table 7. Sensor-modality ablation (mean $\pm$ std; $n = 5$ folds per modality, except audio-RMS at $n = 4$ — one fold's ablation run was unrecoverable). Each row zeros the named sensor channels at the encoder input and re-evaluates the decoder; the encoder is *not* retrained, so this is a lower bound on modality value. A one-way ANOVA across the seven zeroed-modality groups on command accuracy is not significant ($F = 0.07$, $p = 0.998$): the seven modalities are statistically indistinguishable from one another. Every modality versus the full-modality baseline *is* significant (Welch t, $p < 0.001$ in all seven cases), with a uniform $\sim$40–44 pp command-accuracy collapse. Token, type, and parameter-type heads stay within $\sim$4 pp of baseline for every removal.

Modality removed	Token	Type	Command	Param-type	Numeric
baseline (5-fold)	0.781 ± 0.022	0.984 ± 0.009	0.888 ± 0.056	0.993 ± 0.004	0.585 ± 0.033
zero accelerometer	0.743 ± 0.010	0.979 ± 0.010	0.484 ± 0.096	0.978 ± 0.002	0.471 ± 0.008
zero color	0.741 ± 0.010	0.977 ± 0.008	0.457 ± 0.108	0.977 ± 0.004	0.471 ± 0.009
zero electrical	0.741 ± 0.009	0.976 ± 0.009	0.449 ± 0.086	0.978 ± 0.002	0.465 ± 0.009
zero environmental	0.743 ± 0.010	0.977 ± 0.008	0.474 ± 0.077	0.978 ± 0.002	0.470 ± 0.011
zero gyroscope	0.741 ± 0.009	0.979 ± 0.005	0.462 ± 0.049	0.978 ± 0.003	0.469 ± 0.006
zero magnetometer	0.735 ± 0.018	0.975 ± 0.010	0.465 ± 0.084	0.978 ± 0.002	0.466 ± 0.017
zero rms	0.739 ± 0.010	0.978 ± 0.007	0.460 ± 0.088	0.978 ± 0.002	0.465 ± 0.010

The uniform command collapse with a non-significant between-modality ANOVA ($p = 0.998$) is the key result: the inference-time zeroing test confirms all modalities are jointly necessary but cannot rank them. The genuine modality-importance ranking is provided by the leave-one-modality-out encoder retraining (Section 6.11.1).

1044 Interpretation.

The encoder's learned modality-gating routes essential command-discrimination signal through every modality at once; when any one is zeroed at inference, the representation degenerates to one informative enough for the coarse (type, param-type) heads but not for the finer command head. The genuine modality-importance ranking requires an encoder retrained from scratch without each modality, performed in Section 6.11.1, where the retrained encoder fully compensates for any single modality removal (Δ command $\leq$ 0.013; 0/7 Holm-significant). The frozen-encoder "joint necessity" reading is therefore a gating artifact, not a statement of modality information content. The extended interpretation is in Supplementary Materials Section S26, together with the reconciliation with the encoder paper's SHAP-style modality ranking (gyroscope/color/electrical); the decoder-side frozen-encoder ablation does not reproduce that ranking because the two analyses answer different questions.

6.11.1. Per-Modality Encoder Retraining (Leave-One-Modality-Out)

Because the inference-time-zeroing ablation cannot rank modalities (Section 6.11), the deployment-relevant question is whether the encoder, when retrained without a modality, can reallocate capacity onto the remaining channels. We therefore retrain the multi-modal denoising encoder from scratch on the 98-feature base with each modality's channels removed, train a fresh six-head decoder on the resulting frozen encoder, and score all seven cells against a retrained full-stack baseline on the identical held-out test set. A deterministic coverage-repair ordering and a per-cell channel-identity gate guarantee the split is byte-identical across modality exclusions; the full protocol and paired-comparison-validity detail are in Supplementary Materials Section S25.

No measurable single-modality effect.

Retraining the encoder without any single modality produces no measurable change in any decoder head. The full-stack baseline reaches command accuracy 0.904 ± 0.039, and every single-modality removal lands within ± 0.013 of it on the fold-paired effect (electrical largest at -0.013 ± 0.015). The fold-blocked one-way ANOVA across the seven LOMO conditions is non-significant ($F = 0.91$, $p = 0.499$), and the Holm-corrected per-modality paired t-test against zero rejects none (0/7 at $\alpha = 0.05$). Token and numeric heads behave the same way (paired effects within ± 0.011 token, ± 0.022 numeric, all non-significant). Table 8 reports the per-modality numbers; the per-modality paired Δ-command chart is in Supplementary Materials Section S39. The per-modality five-fold means and Hedges' g_z are in Supplementary Materials Section S25; full per-fold Δ, Holm-corrected p, and effect sizes are in `audit/lomo_modality_attribution.json`.

Table 8. Leave-one-modality-out (LOMO) encoder retraining, full-window, five-fold. Each row: the encoder retrained from scratch with that modality's channels removed, then a fresh decoder trained on it (headline recipe). *Command* is the five-fold mean $\pm$ s.d. Δ Command is the *fold-paired* effect — the mean over folds of $\text{command}(m, f) - \text{command}(\text{baseline}, f)$ — which removes the large fold-to-fold variance. p_{Holm} is the Holm-corrected one-sample t-test of those per-fold deltas against zero. Contrast with the inference-time zeroing of Table 7, which holds a frozen encoder fixed.

Modality removed	**Command**	**Token**	**Numeric**	**Δ Command (paired)**	p_{Holm}
Full stack (baseline)	0.904 ± 0.039	0.774 ± 0.024	0.569 ± 0.050	—	—
Accelerometer	0.903 ± 0.037	0.769 ± 0.018	0.559 ± 0.039	-0.001 ± 0.005	1.000
Gyroscope	0.905 ± 0.030	0.772 ± 0.019	0.569 ± 0.041	$+0.000 \pm 0.010$	1.000
Magnetometer	0.896 ± 0.033	0.761 ± 0.018	0.547 ± 0.040	-0.008 ± 0.007	0.407
Color	0.904 ± 0.044	0.773 ± 0.018	0.569 ± 0.038	-0.001 ± 0.013	1.000
Temperature	0.904 ± 0.038	0.775 ± 0.023	0.571 ± 0.049	-0.000 ± 0.015	1.000
Audio-RMS	0.905 ± 0.039	0.779 ± 0.022	0.582 ± 0.043	$+0.000 \pm 0.017$	1.000
Electrical	0.891 ± 0.051	0.776 ± 0.024	0.576 ± 0.050	-0.013 ± 0.015	0.710

Fold-blocked one-way ANOVA across the seven LOMO conditions (on the per-fold deltas): $F = 0.91$, $p = 0.499$. Per-modality paired t-test vs. baseline (Holm-corrected): 0/7 significant at $\alpha = 0.05$. Coverage: 40/40 cells.

Interpretation.

The -42 pp frozen-encoder collapse of Section 6.11 reduces to ≤ 1.3 pp under encoder retraining (none significant). The frozen-encoder gating routes signal through every channel at once, so zeroing one disrupts the routing without changing signal availability; with retraining, capacity reallocates onto the remaining channels. The deployment-relevant statement upgrades from *"the full multi-modal stack is required"* (frozen-encoder result) to *"no single modality is individually load-bearing"* (encoder-retraining result). The orthogonal axis, which physical sensor unit a deployment could ship without, is addressed in Supplementary Section S5.

Orthogonal probes (summary).

Two orthogonal probes corroborate the LOMO main effect above (paired Δ command ≤ 0.013 per dropped modality; fold-blocked ANOVA $F = 0.91$, $p = 0.499$; 0/7 removals Holm-significant), and a body-only reader should not need to open the supplement to see them. *LOSO* (per-physical-sensor-unit leave-one-out under encoder retraining): $\Delta \leq 2.6$ pp per board, no removal Holm-significant, fold-blocked ANOVA $F = 1.04$, $p = 0.415$ (0/6 Holm-significant), with `y_bed__4` statistically indistinguishable from baseline at $p_{\text{Holm}} = 0.975$ (full per-board numbers in Supplementary Section S5). *Nested 6×6 (sensor,modality) interior*: every cell within ± 0.022 pp of baseline, fold-blocked ANOVA $F = 0.694$, $p = 0.896$ (0/36 Holm-significant; Supplementary Section S6). No single-fold interaction-effect pilot (pattern-aware sequence classifier; $K = 451$ 2-digit vocabulary precision; seven-cell window/stride sweep; nested shortcuts$\times$modality and pattern$\times$modality probes; Supplementary Section S8) overturns the modality main effect or the headline. The three nulls (LOMO $p = 0.499$, LOSO $p = 0.415$, nested $p = 0.896$) align: the redundancy is dense rather than a marginal-average effect. The joint LOMO+LOSO deployment statement is consolidated in Section 7.2.0.1.

6.12. RQ4: Cross-Class Generalization (Leave-One-Class-Out)

For each of the nine operation classes (`adaptive`, `adaptive150025`, `face`, `face150025`, `pocket`, `pocket150025`, `damageadaptive`, `damageface`, `damagepocket`), we train the decoder without that class in the training partition and evaluate on its held-out test samples. This stress-tests whether the decoder generalizes across machining operations or memorizes per-class shortcuts.

Each holdout removes every file of the held-out class from the fold 1 train/validation partitions, re-stratifies the remaining 8-class training partition at the headline 80/20 split, and tests on the entire population of held-out-class files. The coverage-repair step is deliberately *not* re-run, leaving the open-vocabulary residual the protocol is designed to expose. All hyperparameters are reused verbatim and each holdout is trained from scratch; the full partition-construction detail is in Supplementary Materials Section S27.

Across all nine LOCO holdouts (one fold per holdout, fold-1 file split): token accuracy mean 0.501 (min 0.29, median 0.53, max 0.61); sequence accuracy mean 0.001 (range $[0, 0.005]$); command accuracy mean 0.213 (min 0.000, median 0.21, max 0.678); numeric accuracy mean 0.152 (min 0.000, median 0.16, max 0.402). Type and parameter-type accuracy across the nine holdouts are markedly more robust (means 0.883 and 0.945 respectively; spreads ± 0.061 and ± 0.036 across classes, subject to the same single-fold caveat below).

Two scope qualifications govern these numbers. First, the nine operation classes are the population of operation classes in the corpus, not a sample from a class population, so we report the finite-population descriptive distribution rather than between-class parametric/bootstrap CIs and make no inferential claim about a hypothetical wider class population. Second, the per-holdout numbers are single-fold (file-split fold 1 only), so the open-vocabulary headline command mean 0.213 is a single-fold result; the 9-class spread $\sigma = 0.191$ reflects between-class heterogeneity, not within-class fold-to-fold sampling.

Relative to the in-distribution 5-fold baseline (0.781/0.026/0.888/0.585), the LOCO setting incurs a $\sim$28 pp drop on token accuracy, a $\sim$3 pp drop on sequence accuracy, a $\sim$67 pp drop on command accuracy (mean), and a $\sim$43 pp drop on numeric accuracy. We report this contrast as a descriptive gap rather than a two-sample test: the five in-distribution folds and the nine LOCO holdouts are not two random samples from a common population (file-stratified CV and enumeration over operation classes are different experimental designs), so a Welch or Mann–Whitney test on them tests nothing well-defined. The central evidence is the per-holdout collapse itself (a command-mean gap of 0.675); the corresponding 5-fold and 9-class per-cell ranges do not overlap on command or numeric.

Per-holdout variance and the `damagepocket` outlier.

The nine per-holdout command accuracies span $[0.000, 0.678]$ with $\sigma = 0.191$. Eight of the nine holdouts cluster in $[0.000, 0.273]$, roughly in line with the mode-collapse story below; one holdout, `damagepocket`, reaches command accuracy 0.678, token 0.548, and numeric 0.402. The structural reason is shared motion vocabulary: the `damagepocket` test class shares most of its motion vocabulary (predominantly G1 linear cuts with modest G3 finishing arcs) with the `pocket` and `pocket150025` variants retained in training, so the model transfers its in-distribution pocket-pattern command discrimination to the held-out damaged-pocket samples with only modest degradation. By contrast, `damageadaptive` (command 0.000) and `damageface` (command 0.094) do not benefit similarly from their parent operations, because the adaptive and face motion patterns differ more strongly between damaged and non-damaged variants than the pocket variants do. Cross-class generalization is therefore not uniformly absent: it succeeds when the held-out class shares motion vocabulary with retained training classes and collapses otherwise. The median across the nine holdouts (0.21) better reflects the typical cross-class behavior than the mean.

Per-class LOCO command F1.

The collapse spans every command class: the model defaults to the modal G1, the mode-collapse mechanism (Section 7.3) under an unseen motion vocabulary. The per-class F1 breakdown is in Supplementary Materials Section S27. The heads degrade asymmetrically. Type and parameter-type survive LOCO ($-9/-5$ pp), so the encoder preserves gross instruction structure even for a novel class; command and numeric require the in-distribution operation pattern. The deployment-realistic claim for an instruction-level monitor is therefore bounded by these LOCO numbers, not the 5-fold headline (the open-vocabulary counterpart of the closed-vocabulary regime, Section 8.1). *Scope binding.* The deployment claim of this paper is explicitly scoped to in-distribution post-hoc audit, closed-world over the G-code operation classes observed at training time. Open-world / novel-operation-class deployment is out of scope on the present system and is deferred to future cross-platform replication (Section 8). Within the closed-world envelope the operating point is the in-distribution command-swap point of Section 7.4; outside it the system is not a usable detector (LOCO command-swap FPR 0.52, sign-flip and feed-edit at or below chance). The full per-holdout, per-head breakdown chart is in Supplementary Materials Section S39.

6.13. Noise Augmentation

Gaussian sensor noise ($\sigma = 0.02$) plus feature-dropout and token-masking during training is a clean *negative* result. Token (0.732) and numeric (0.448) accuracy are preserved, but command drops to 0.331 ± 0.076, a $\sim$56 pp shortfall against the 0.888 headline, because the regularization pushes the command head toward the modal class. We do not recommend this configuration here; the full numbers are in Supplementary Materials Section S38.

6.14. End-to-End Seq2Seq Baseline: What the Frozen Encoder + Grammar Mask Buy

Motivation.

The headline architecture stacks a frozen MM-DTAE encoder (Section 4.2), a grammar-constrained autoregressive decoder, and a six-head structured output on top of an otherwise standard Transformer decoder. None of these components is individually novel; the question is whether the stack earns its keep against a vanilla seq2seq trained end-to-end on the same data. This subsection answers it.

Baseline.

The baseline is a from-scratch Transformer encoder–decoder (8.64 M parameters, a same-order-of-magnitude budget to the frozen encoder's 8.6 M: 4-layer encoder + 4-layer decoder, $d_{\text{model}} = 256$, 8 heads) trained directly on (sensor window $\rightarrow$ G-code token sequence) with cross-entropy. It uses no frozen encoder, no grammar mask, no six-head structured output, no scheduled sampling, and no auxiliary losses. It shares the V8 vocabulary, the full-window target formulation, the file-level 5-fold CV, and the evaluation metrics. Code: `scripts/evaluation/run_seq2seq_baseline.py` and `src/miracle/model/seq2seq_baseline.py`.

Result.

The head-to-head table is in Supplementary Materials Section S39; three findings follow (extended interpretation in Supplementary Materials Section S30). *(1) Teacher-forced command accuracy is architecture-invariant.* The baseline reaches 0.877 ± 0.031 TF command accuracy, indistinguishable from the headline 0.888 ± 0.056. *(2) The +76.6 pp command / +77.9 pp parameter-type AR gap is a metric-construction artifact, not autoregressive-decoder superiority.* The baseline reads command identity off the autoregressively generated token stream and collapses to 0.122 ± 0.051, the most-frequent-class prior, whereas the headline command head is a non-autoregressive classifier on the encoder memory (TF/AR-invariant at 0.888). Compared like-for-like on the AR token stream, the two architectures are within a fold of each other (token 0.154 vs. 0.215; numeric 0.100 vs. 0.104). *(3) TF numeric reverses.* The baseline's TF numeric accuracy (0.754 ± 0.053) exceeds the headline's bucketized digit-decomposition output (0.585 ± 0.033) by +16.9 pp, a future-work item for the numeric head that does not translate to an AR-mode deployment advantage (AR numeric 0.100 vs. 0.104).

Interpretation.

The contrast scopes the paper's architectural contribution narrowly. The +77 pp AR command/parameter-type gap is the consequence of routing categorical prediction off the autoregressive token stream onto the encoder memory, not a claim of autoregressive-decoder superiority; read as a like-for-like AR comparison, the two architectures' token and numeric streams collapse together. The contribution is therefore the design choice of harvesting the categorical fields with non-autoregressive classifier heads, which is robust to the exposure-bias collapse that limits any AR token reader on this corpus. The comparison explicitly does *not* claim a TF-mode architectural advantage on the categorical heads, and it concedes a TF-numeric disadvantage attributable to the bucketization design. This scoping matches the post-hoc audit framing of Section 7.2.0.1: the categorical heads carry the deployment signal, and they do so independently of autoregression by construction.

7. Discussion

7.1. What is recoverable, and the class-prior boundary

What is recoverable, and why.

Recoverability factorizes into three structural tiers (numbers in Section 6.2 and Table 4). *Categorical* structure (command and axis identity) is the strongest tier. The sensor pathway determines which class of motion is occurring and which axes are involved, independently of file position; the metadata ablation (Section 6.10.1) confirms that positional features add nothing.

Motion sign is recovered with an axis asymmetry: Y near-perfect, X and Z lower. On this geometry the result suggests the $\pm X$ and $\pm Z$ sensor signatures are less separable than $\pm Y$, plausibly because of sensor-board layout relative to the workpiece origin or X-symmetric workpiece geometry. A workpiece-rotation experiment would isolate the cause.

Numeric precision is the limiting tier, and the limit is information-theoretic. Loss concentrates at the middle (fine-value) digit positions (Section 6.4). A 64-second window observes only relative motion, not absolute machine coordinates; recovering the latter requires positional context (deliberately withheld) or longer-range autoregressive context that the per-row formulation does not provide. The failure-mode analysis is in Section 6.9. Whether the categorical tier is genuine sensor recovery or the encoder's class prior is the subject of the next subsection.

Is categorical recovery sensor-driven or operation-class structure?

The frozen encoder was trained for nine-class operation classification on the *same* file-level fold partitions used here (Supplementary Materials Section S21). Its memory is low-rank and linearly separates the operation classes (Section 4.5). The headline command accuracy could therefore be operation-class structure re-expressed at the command level rather than genuine per-instruction sensor recovery. We quantify this with a class-prior control (`audit/confound_class_prior.json`; `scripts/analysis/confound_class_prior.py`): at every COMMAND-token position of the full-window predictions, the decoder is compared against (i) a *class-prior* predictor that emits the modal command of the window's operation class (modal computed on the fold's training split) and (ii) a within-window modal predictor.

The decoder's command accuracy at command positions is 0.795 ± 0.121 (5-fold), versus a class-prior baseline of 0.572 ± 0.004 and a within-window-modal baseline of 0.587 ± 0.003. The 0.795 is command accuracy evaluated only at COMMAND-token positions of the full-window predictions, a different position subset and aggregation than the 0.8875 head accuracy in Table 1; the two are consistent measurements of the same predictions, not a discrepancy, exactly as for the per-file mean in Section 8.4. Under file-level cross-validation the operation-class modal is the strictest legitimate sensor-independent null: a per-file modal predictor that uses only training data necessarily reduces to the operation-class modal, because each test file is unseen and its file-specific modal cannot be estimated from training data alone.

The $+22$ pp increment over that null is a 4-of-5-fold effect, not a uniform one. On fold 1 the decoder (0.552) is at or below its own class prior (0.568, a -0.016 delta; fold 1 is the covariate-shift outlier of Section 8.3.0.1), while folds 2–5 sit $+0.27$ to $+0.30$ above the prior (0.85–0.86 decoder vs. ~ 0.57 prior). The paired-by-fold contrast gives a mean increment of $+0.222$ (per-fold deltas $\{-0.02, +0.28, +0.27, +0.30, +0.28\}$, 4/5 positive). The mean is driven entirely by the four non-outlier folds and would not survive removal of more than one fold. At $n = 5$ we report the increment as an effect size with its per-fold scatter rather than a p-value, because no five-point test can confirm an increment whose sign flips on one fold. A percentile bootstrap over the five per-fold deltas gives a 95% CI of approximately $[+0.10, +0.29]$. The fragility is a leave-one-fold effect, not a bootstrap-edge one: jackknife leave-one-out means are $+0.20$ to $+0.21$ when any of folds 2–5 is dropped and $+0.28$ when the fold-1 outlier is dropped (folds 2–5 mean $+0.282$). The magnitude should be read as an *upper* bound on the sensor-driven within-class signal (see the class-conditional-modal bound below).

A majority of the headline command signal (roughly 0.57 of the 0.79) *is* therefore attributable to operation-class structure the encoder already encodes. The absolute command numbers must be read in that light; this is the representational counterpart of the leave-one-class-out collapse (Section 6.12). On top of that prior, the decoder adds the within-class increment characterized above on the $97.4\% \pm 1.4\%$ of windows that contain more than one distinct command. Its accuracy on those command-heterogeneous windows, 0.794 ± 0.122, is indistinguishable from its pooled accuracy, so it is not merely echoing a per-window stereotype. The honest statement is therefore neither "the decoder recovers commands from sensors" nor "the decoder only re-reads the class label": the decoder adds a bounded, fold-dependent increment of per-instruction command information on top of a large operation-class prior.

This increment is an UPPER bound (closed-vocabulary regime, frozen-encoder representation reuse), but it is robust to conditioning the null on operation class. A *class-conditional* modal predictor (the most-frequent command *within* each operation class), scored on the same all-command-position basis as the decoder, reaches only 0.576 ± 0.003 (5-fold; `audit/class_conditional_modal_5fold.json`), essentially identical to the 0.572 unconditional operation-class modal. Conditioning the null on operation class therefore does not raise it, and the $+22$ pp increment is *not* shrunk by this stricter null. (A first-command-position-only class-conditional modal reaches ~ 0.81, `audit/class_conditional_modal.json`, but that is an easier estimand scored on a single position per window, not a like-for-like null for the decoder's all-position accuracy.) The $+22$ pp figure remains an upper bound on per-instruction sensor-driven recovery because of the closed-vocabulary regime and frozen-encoder reuse, not because a class-conditional null beats it.

A per-file-modal control (raised in review) would tighten this bound further given the 97.6%/99.0% verbatim line overlap (Section 8.1), but it is not computed for this release. A per-file-modal predictor exceeding 0.572 requires test-file labels, so the $+22$ pp lift is already measured against the strictest legitimate train-only null, and the per-file-modal control is best read as an oracle-style upper bound (full argument in Supplementary Materials Section S21).

7.2. Sensor-stack redundancy and deployment recommendations

Sensor-modality importance and deployment implications.

A cost-reduced post-hoc audit installation is viable on this platform: drop `y_bed__4`, retain `spindle2`. The same redundancy applies whether the system is deployed as an offline forensic audit complement or a periodic process-verification routine. Three results support the recommendation (full version in Supplementary Materials Section S32). The frozen-encoder inference-time ablation establishes *joint necessity* (40–44 pp uniform collapse, between-modality ANOVA $p = 0.998$). The encoder-retraining LOMO complement (Section 6.11.1) shows no single modality is individually load-bearing (paired Δ command ≤ 0.013; $F = 0.91$, $p = 0.499$), so the frozen-encoder collapse is a gating artifact. The LOSO study (Supplementary Section S5) shows any single physical sensor board costs $\leq$2.6 pp (largest `spindle2` -0.026; `y_bed__4` ≈ 0 at $p_{\text{Holm}} = 0.975$), and the nested 6×6 interior (Supplementary Section S6) is flat at ± 0.022 pp ($F = 0.694$, $p = 0.896$). The redundancy is dense: any single sensing modality can be dropped at $\leq$1.3 pp expected cost and any single board at $\leq$2.6 pp.

Comparison with side-channel recovery in additive manufacturing.

Additive-manufacturing side-channels [16–18,20,21] reconstruct full print geometry. We match their categorical fidelity but not their geometric fidelity. The gap is expected: the AM literature samples raw acoustics at kHz–MHz, whereas the present 4-Hz stack is deliberately low-rate (Section 2.3, Section 8.2). The comparison bounds what the low-rate stack forgoes; it does not indicate a deficiency of the method.

When to use this decoder vs. alternative approaches.

The decoder is best suited to three deployment patterns, all offline (full descriptions in Supplementary Materials Section S32). *(1) Post-hoc audit (primary):* command/axis-letter disagreements against the controller log flag specific rows for offline analyst review on contested runs; this is the posture for which the operating point of Section 7.4 was characterized. *(2) Offline process verification (secondary):* batched per-row predictions are reconciled against the commanded program as an end-of-shift step, with the measured-ρ $K = 10$ point (Section 7.4) as a human-in-the-loop triage aid only. *(3) Anomaly localization (tertiary):* command-level deviations localize session-level anomaly flags [6,15] to specific G-code lines. The decoder is explicitly not an autonomous real-time alerting tool. Numeric-value precision is too low for coordinate reconstruction throughout; command/axis/sign-level analysis is the reliable signal within the trained toolpath envelope.

Positioning: technology readiness and target verticals.

The work characterised here is at TRL 4: component validation in a laboratory environment on a single desktop CNC platform (a frozen-encoder + Transformer-decoder stack under file-level cross-validation). Cross-platform replication on a production vertical machining centre is the prerequisite for advancing to the TRL 5–6 cross-platform target and is explicitly future work (Section 8.6). The target adoption pipeline is regulated discrete-part manufacturing with auditable part provenance: FDA 21 CFR Part 820 (medical), AS9100D §8.5.2 (aerospace), and IATF 16949 §8.5.2.1 (automotive). In these settings a controller-independent record of executed instructions complements the existing controller log and the file-hash / attestation lines of defense. The long-run modernization path for the G-code interface itself is ISO 14649 (STEP-NC) [66], which makes program-level intent machine-checkable; the present sensor decoder is a back-compatible monitor for the installed G-code base during that transition.

Deployment-time target mode: prefer per-row.

In autoregressive deployment the per-row target mode (each window predicts the single G-code line that fired within it) is preferable to the full-window mode used for training-time analysis. Per-row predictions are short (5–15 tokens) and independent across rows, and are consequently robust to the autoregressive mode-collapse that affects full-window long-sequence generation (Section 7.3). Categorical-head accuracy, the primary signal for the three deployment patterns above, is unaffected by target mode because the categorical heads are independent classifiers on the encoder memory rather than autoregressive token predictors. The full-window mode remains the natural training-time configuration for studying within-window row-disambiguation, but it should not be used at deployment time when the goal is reliable per-instruction recovery.

Conversely, the decoder is *not* a stand-alone replacement for the controller log (no exact-line reproduction). It is not yet suitable for sub-millimeter toolpath verification (numeric precision is insufficient). The combination of the present decoder with the encoder paper's operation classifier provides a layered verification stack: the encoder flags session-level operation-class anomalies, and the decoder localizes them to specific G-code lines.

Differentiation from file-level program audit and controller attestation.

The orthogonal-defenses argument is stated once, as the canonical threat model, in Section 3.5 ("Adversary model: orthogonal defenses") and is not re-derived here. That argument covers why the sensor decoder runs in parallel with file-hash signing [2,54,67] and TCB controller attestation [53] rather than in place of them, and which residual attack subset (runtime substitution, selective skip/duplicate, wrong-workpiece-or-fixturing) the sensor line uniquely addresses.

Configuration choices studied (not a deployment endorsement).

Supplementary Materials Section S28 tabulates the configuration options this study evaluated and the rationale for each (training schedule, numeric vocabulary, target mode, positional metadata, grammar layer, calibration, $K = 10$ aggregation, sensor stack, inference platform, and detectable attack classes). The categorical-recovery rows (target mode, grammar layer, calibration, sensor stack) are cross-validated study findings; the aggregation and attack-class rows are bounded by the in-distribution closed-vocabulary envelope and must be read with the measured-ρ, prior-driven, and LOCO qualifiers of Section 7.4.

7.3. Autoregressive Mode Collapse on Full-Window Targets

The autoregressive decoder converges toward a corpus-modal facing-pattern toolpath. Across the 549-sample 5-fold test split it produces 407 distinct strings (vs. 459 for ground truth), and the modal predicted string appears in 9.5% of windows vs. 2.9% for the modal ground-truth, a 3.3× over-representation (`audit/ar_output_diversity.json`). The collapse is therefore *partial*: the decoder does not produce a single canonical output, but it over-concentrates on the modal facing-pattern at roughly three times the natural rate. This mode collapse is the mechanism behind the autoregressive-token-accuracy drop reported in Section 6.2; the categorical heads are unaffected for the reason stated in Section 3.4. The target-length correlation (Pearson $r \approx -0.15$) is weak, and we treat it as corroborating only, not load-bearing. Section 7.2.0.3 recommends per-row mode at deployment time on this basis.

A worked three-sample illustration of the modal-trajectory collapse and the pairwise prediction-similarity heatmap (Supplementary Figure S3) are in Supplementary Section S4.

7.4. Threat Model Detectability (Results and Discussion)

The trust boundary and the five attack classes are formalized in Section 3.5. This section provides the per-attack-class detectability characterization, the operating-point analysis, and the sensor-side adversary discussion. The measured operating-point results (Table 9, Supplementary Figures S13/S14) are reported here rather than in Section 6 because they are interpretable only against the trust boundary and attack-class taxonomy of Section 3.5 and the per-field recoverability factorization of Section 6.2. We follow the manufacturing-cybersecurity literature [1,3,4,6,53,54] throughout.

Scope boundaries (sensor-side, upstream, and out-of-distribution adversaries).

Three adversary classes fall outside the present detector's scope and are enumerated with their operational mitigations in Supplementary Materials Section S33. (i) A *sensor-side* adversary (wire-tap, EMI injection, telemetry packet-replay, calibration spoofing) can defeat the monitor by attacking the sensor path before the decoder; none of these attacks is benchmarked here, and a deployment that does not enforce the listed mitigations should not invoke the "physics-of-record" framing. (ii) An *upstream CAM/post-processor* adversary whose substituted toolpath stays in-vocabulary is invisible to the decoder, which agrees with the already-malicious controller stream; the present system is an execution-integrity monitor, not an upstream-design-integrity monitor. (iii) *Out-of-distribution* attacks (novel command tokens, novel operation classes) have no calibrated false-alarm rate and collapse to the LOCO operating point (command-swap FPR 0.52).

Detection capability by attack class.

The per-field recoverability characterization maps onto the attack classes as follows (full per-class detail in Supplementary Materials Section S33). *Command-identity* (G0→G1) and *axis-identity* (X→Y) substitutions are *detectable* (command accuracy 0.8875 ± 0.0558, parameter-type 0.9931 ± 0.0036). *Motion-sign* substitution is *partially detectable*: only Z-sign carries non-trivial sensor signal, and Y/X sign "detection" is prior mismatch ($H_Y^{\text{sign}} = 0.08$, $H_X^{\text{sign}} = 0.38$ vs. $H_Z^{\text{sign}} = 1.04$ bits). *Numeric-value* tampering is *not detectable at fine precision* by an information-theoretic bound (thousandths source entropy ≈ 3.3 bits, slot-4 accuracy 0.42). *Feed-rate / spindle-speed* tampering is *not evaluable* on the present corpus. *Replay* is detectable indirectly (argued, not benchmarked). *Skip/duplicate* at per-row granularity is not directly addressed.

Illustrative tamper operating points (prior-dominated; *not* a validated detector)

The operating points in this subsection are illustrative, not a validated detector, and the reader should treat them as such. No physical tamper was performed. The "tamper" is a synthetic relabelling of the honest predictions, and the reported false-alarm rate is mechanically the recoverability complement (one minus per-field recoverability) on the same population, not an independent honest-run-pair measurement (Table 9 footnote). A genuine independent false-alarm estimate is computable in principle from the multi-file-per-class corpus, by scoring the decoder's prediction on one honest run against the controller log of a *different* honest run of the same operation class. That estimate requires per-file decoder inference not persisted in the released artifacts and is left to the artifact roadmap; the recoverability-complement rate is the disclosed proxy here. The detection signal is also *prior-dominated* (quantified below). We report the numbers because they make the per-field recoverability characterization concrete in attack terms. They do not constitute a deployable detector.

First-pass tamper-injection operating points (illustrative).

We ran a synthetic tamper-injection audit on the 5-fold full-window predictions under autoregressive deployment-true decoding with the FSM grammar layer enabled (Section 4.3.2). For each held-out sample we form the honest pair $(g_{\text{true}}, g_{\text{pred}})$ and mutate g_{true} by command swap, motion-sign flip on the first signed axis, or feed-rate $\pm 10\times$ edit; a tamper is flagged when the relevant token-class in g_{pred} differs (thresholds: exact match for command/sign, $> 50\%$ relative deviation for F; $n = 483$ command-swap, 549 sign-flip, 111 feed-edit pooled). Table 9 reports the ROC-AUC and threshold-matched operating point per attack class (baseline / with-metadata) under both the in-distribution closed-vocabulary and the open-vocabulary LOCO regimes; the pattern is consistent across both. Command-swap is moderately discriminable. Sign-flip and feed-rate detectors collapse to near-chance under AR (sign-flip because 99%-positive Y dominates the pooled accuracy by prior; feed-rate because the 4-Hz stack cannot resolve feed-rate transients) and are non-detectable under LOCO.

Table 9. Tamper-detection discrimination summary and threshold-matched operating point by attack class, regime, and configuration (“Baseline” = no positional metadata; “Metadata” = window-index/source-file exposed; AR deployment-true decoding with FSM grammar). Command-swap rows report a true swept-threshold ROC-AUC with Hanley–McNeil 95% CI [68] ($n_{\text{cmd}} = 483$); sign-flip and feed-rate rows report the single-threshold balanced-accuracy estimator $\text{BA}{=}\frac{1}{2}(\text{TPR} + (1 - \text{FPR}))$ (AUC undefined for exact-match/threshold-deviation detectors; BA < 0.5 is anti-discriminative). FPR is the decoder-vs-controller-log disagreement rate on the unmutated honest pair — mechanically (1 minus per-field recoverability), the recoverability complement re-labelled as an alert rate, not an independent honest-run-pair measurement (smallest empirically-resolved FPR $\approx 1/483 \approx 2.1 \times 10^{-3}$). Sources: `audit/threat_model_tamper_AR_5fold_FSM.json`, `threat_model_tamper_loco.json`.

Attack class	Config.	AUC or BA [95% CI]	TPR	FPR
In-distribution closed-vocabulary regime (5-fold pooled)				
Command-swap (G0→G1) [AUC]	Baseline	0.80 [0.77, 0.83]	0.921	0.299
Command-swap [AUC]	Metadata	0.86 [0.84, 0.88]	0.948	0.222
Sign-flip ($X+ \to X-$) [BA]	Baseline	0.68	0.703	0.350
Sign-flip [BA]	Metadata	0.78	0.798	0.244
Feed-rate $\pm 10\times$ [BA]	Baseline	0.61	0.216	0.0055
Feed-rate $\pm 10\times$ [BA]	Metadata	0.76	0.532	0.0146
Open-vocabulary leave-one-class-out (LOCO) regime; BA$=\frac{1}{2}(\text{TPR} + (1 - \text{FPR}))$				
Command-swap [BA]	Baseline	0.66	0.847	0.519
Sign-flip [BA]	Baseline	0.20 (anti-disc.)	0.138	0.748
Feed-rate $\pm 10\times$ [BA]	Baseline	0.50 (degenerate, $n = 45$)	0.000	0.000

The pooled sign-flip TPR/FPR mixes prior-driven detection on Y with sensor-driven detection on X and Z; on Y the 99%-positive prior alone explains essentially all of the head’s accuracy ($H_Y^{\text{sign}} = 0.08$ bits). The per-axis decomposition above and Section 6.4 should therefore be read alongside the pooled operating points. Autoregressive mode collapse strips per-axis-sign and per-feed-rate information, and the pessimistic reading of the sign-flip and feed-rate rows is their alert-budget cost. The ROC-space operating points are shown in Supplementary Figure S14.

Adding positional metadata strictly improves tamper detection on every attack class (Table 9; the feed-rate TPR improves $\sim 2.5\times$). This is the security counterpart to the mode-collapse-mitigation finding in Section 6.10.1: the per-window anchor that breaks the corpus-modal trajectory also makes the window-specific predictions informative enough to discriminate tampered from honest controller logs. For security-oriented use the with-metadata configuration is therefore preferable, with one caveat (Section 7.4, positional-metadata row): window-index/source-file metadata is controller/pipeline-derived while the controller is the untrusted element, so its exposure to the monitor is a residual trust-boundary tension, not an unqualified recommendation. For use cases where positional metadata is unavailable, or where a clean “sensors-only” baseline is needed, the no-metadata configuration is the conservative choice.

False-alarm cost.

At the measured median row duration of 0.25 s, one CNC on an 8-hr shift commands $\sim 1.15 \times 10^5$ row decisions. The with-metadata in-distribution FPR 0.222 therefore yields $\sim 2.5 \times 10^4$ alerts/shift/machine (open-vocabulary LOCO FPR 0.519: $\sim 6 \times 10^4$), two to three orders of magnitude above the SOC tier-1 alert-budget guidance of $\leq 10^2$ per analyst per shift implicit in NIST SP 800-61 Rev. 2 [69]. A row-level monitor is therefore not viable as a standalone SOC interface without an aggregation or triage stage, and the deployment recommendation below accordingly anchors on the lowest-FPR post-hoc audit operating point, not this 0.222 alert rate. The candidate mitigations (sliding-K aggregation, fixed-low-FPR operating-point selection, temperature-scaled calibration) and the extended false-alarm-cost, aggregation, measured-autocorrelation/prior-control, and adaptive-adversary discussion are in Supplementary Materials Section S31; none of the mitigations closes the gap.

Operating-point recommendation.

Combining the alert-budget analysis above with the per-class detectability map, the recommended deployment posture for the present system is **forensic-support post-hoc audit, explicitly not real-time SOC monitoring** (the posture of Sections 1 and 3.5). We recommend two concrete operating points and explicitly do *not* recommend a standalone row-level real-time deployment. **(1) Post-hoc forensic / audit:** operate the with-metadata command-swap detector at the lowest empirically-resolved FPR, approximately $\mathrm{FPR} \approx 2 \times 10^{-3}$ on the $n_{\mathrm{cmd}} = 483$ ROC. This is not the previously stated 10^{-3} target, which extrapolates below the empirical resolution and would require a confirmatory larger held-out honest population; TPR is $\sim$ 0.20–0.30 at that point on Supplementary Figure S14, in-distribution. The low row-level TPR is acceptable in the post-hoc audit context, where the operator is reviewing a contested run rather than adjudicating a live stream and the detector's job is to surface command-class disagreements. **(2) Human-in-the-loop confirmation at $K = 10$ majority:** adopt the measured-ρ $K = 10$ majority point (quantified below) only as a triage aid to a human reviewer, not as an autonomous alert. Both points hold only within the in-distribution closed-vocabulary envelope. Under the open-vocabulary LOCO regime the system is not a usable detector, and the recommendation reduces to forensic post-hoc inspection of categorical disagreements alone.

The deployment-relevant security numbers, established in full in Supplementary Materials Section S31, are three: measured autocorrelation, prior-dominance, and open-vocabulary collapse. (1) At the empirically measured honest-trace autocorrelation $\rho = 0.42 \pm 0.22$, the $K = 10$ majority command-swap point is with-metadata FPR 0.098 at TPR 0.97 (baseline FPR 0.184 at TPR 0.94), *not* the optimistic $\sim$ 0.04 that the false row-independence assumption gives. (2) The alert is largely prior-driven: the sensor-conditioned decoder adds only $\sim$ 8 pp of TPR over a sensor-*independent* corpus-modal (modal-substitution) predictor, and the stricter class-conditional command-accuracy null does not absorb this lift either (Section 7.1.0.2). (3) Under the open-vocabulary LOCO regime the command-swap detector collapses to FPR 0.52 at TPR 0.85, with the sign/feed detectors below chance (Table 9); the LOCO point is the security-relevant operating point, and the in-distribution K-aggregated figures are an optimistic in-envelope ceiling.

The feed-rate detector's low FPR (0.005–0.015) is an *in-distribution* artifact and does not translate to a usable detector (TPR 0.00 under the open-vocabulary LOCO regime; Table 9). The feed rate is also the canonical adaptive-adversary bypass field, and recovering it is not a current capability of the 4-Hz stack.

Adaptive adversary (a structural bypass, not a caveat).

The detector's blind spots are not incidental; they define a complete bypass for any adversary who knows the scheme. The recoverable fields are command identity, axis presence, and motion sign; the *un*-recoverable fields are numeric coordinate value and feed rate (Section 6.3). An adversary who keeps the command/axis/sign envelope fixed and corrupts only coordinates or feed rate is therefore *invisible by construction*. No K-row aggregation helps, because the per-row signal on those fields is already at chance. This is exactly the high-value attack class the Introduction motivates (wrong depth of cut, wrong feed, a scrapped part or damaged tooling). The residual security value against a non-naive adversary is therefore small: the monitor meaningfully constrains only command/axis/sign substitutions that also stay within the trained toolpath envelope. We state this as a structural limitation, not a tunable parameter (extended discussion in Supplementary Materials Section S31).

Limitations of the verification.

The decoder cannot detect attacks whose physical effect is indistinguishable from a benign program at the 4-Hz sample rate; sub-second motion details (rapid clearance moves, small finishing passes) may be temporally aliased. Mitigations (higher sample rate, additional sensor modalities, encoder retraining with row-level supervision) are open directions (Section 8). None of them closes the adaptive-adversary bypass above, which structurally requires recovering the numeric/feed fields the present sensor stack does not carry. A separate float-epsilon target-encoding artifact in the numeric-digit pipeline (characterized in the companion limit paper [33] and closed by a patch on the development branch) could in principle let an adversary aware of the historical encoding craft "corrected" G-code to trigger false alarms on a monitor trained on buggy targets; the patch closes this narrow detection-mismatch path for future deployments.

Audit leverage by toolpath entropy.

The per-attack-class detectability above carries a defense-in-depth corollary along an operation-class entropy axis, developed in full in the companion limit paper [33]: sensor-side audit is most effective on low-entropy operation classes (facing overruns, drilling cycles), is information-theoretically blind on high-entropy adaptive optimizer-generated toolpaths, and complements rather than replaces cryptographic file-hash signing.

8. Limitations and Future Work

8.1. Dataset Limitations

Continuous-parameter coverage.

The near-absence of variation in feed rate (F), spindle speed (S), and arc parameters (I/J) is the most consequential dataset limitation. Table 3 reports each field's positive support: F appears in $<1\%$ of per-row samples, S is essentially absent from both modes, and I and J never appear (arcs use R notation only). The accuracy numbers for these fields in Table 4 are therefore artifacts of their near-zero baseline support, not evidence of sensor-driven recovery. Addressing these gaps would require a designed factorial data-collection campaign that varies feed rate, spindle speed, depth of cut, and material at a higher sampling rate (Section 8.6).

Material variety and single desktop-platform scope.

All recorded runs use a single workpiece material on a single Bantam Tools desktop mill (cast-aluminum frame, stepper drive, no flood coolant, dry cutting on machinable wax). Material-class generalization therefore cannot be tested on the present corpus; testing it would require a multi-material corpus (e.g., aluminum, steel, Delrin). *Cross-platform generalization is the larger open question.* A production VMC differs from the desktop platform along every sensing axis quantified in Section 5.1 ("Platform characterization"), and no result in this paper has been replicated on a production-class VMC. Until cross-platform replication is performed (Section 8.6, item (d)), every recoverability claim, deployment recommendation, and detectable-attack-class guarantee, together with the entire forensic-support posture, must be read as conditional on this single platform/material envelope. The results constitute a characterization study *on that envelope*, not a portable deployment claim.

What transfers and what does not.

The findings split into platform-agnostic and platform-specific components. The AR mode-collapse mechanism, the structured-vs-numeric recoverability factorization, the class-prior decomposition framework, and the seq2seq architectural-contribution scoping are mechanism/theory findings expected to hold under any low-rate multi-modal stack and Transformer decoder. The headline numbers, the 6-sensor 7-modality inventory, the machinable-wax signature, the 4 Hz rate, and the 9-class taxonomy are platform-specific quantitative results expected to transfer with adjustment. One transfer claim is flagged mechanism-uncertain. The categorical-recovery result is platform-agnostic only at the level of the factorization, not the recovery mechanism, because whether the same modalities carry the command signal on a servo-driven, flood-cooled, ferrous-workpiece VMC is unverified. The full transfer matrix is in Supplementary Materials Section S37.

Label set scale.

The corpus is dominated by a small set of frequently-recurring G-code lines, with G1 strongly dominant and a long low-frequency tail. Each fold contains of order 10^3 distinct lines but only a handful of distinct motion-command classes (Section 8.1 reports the exact per-fold distinct-line counts and the train/test overlap). Per-class metrics are macro-averaged over that tail. Results for G3, M30, and arc parameters would therefore benefit from substantially more diverse data.

Verbatim train/test line overlap (closed-vocabulary regime).

The file-level 5-fold splits guarantee file-level disjointness between train and test, but the coverage-repair procedure (Section 5.3; Supplementary Materials Section S20) ensures by design that every G-code line in the test partition has been seen during training of that fold. The measured line-level overlap (`audit/train_test_gcode_overlap.json`) is 97.6% of distinct test lines and 99.0% of test line-instances in per_row mode (5-fold average), and 97.9%/99.1% in full_window mode. The 5-fold experiments therefore evaluate the mapping of a sensor signal to one of the in-vocabulary G-code lines, not the generation of novel lines, and the genuinely open-vocabulary evidence is the LOCO protocol of Section 6.12, under which command accuracy degrades from 0.888 to 0.213 ± 0.191 and numeric accuracy from 0.585 to 0.152.

8.2. Methodological Limitations

Frozen-encoder representation.

The encoder was trained on the same fold splits as the decoder. Its training objective (nine-class operation classification) includes no row-level G-code supervision. Both factors may limit the encoder's preservation of fine-grained per-row sensor structure. Retraining the encoder with row-level supervision and re-running the headline 5-fold would quantify the residual encoder-side contribution. This is the principal open direction (Section 8.6).

Sampling rate.

The 4 Hz rate is not a marginal concern. The row-execution-duration audit (21,541 instances; `audit/gcode_row_durations.json`) shows that the median row executes in a single 0.25 s sample and that 77.7% **of instances occupy** ≤ 1 **sample period** (83.1% ≤ 0.5 s, 86.3% ≤ 1 s). Most individual instructions are therefore sub-sample events the stack cannot resolve as distinct transients. What *is* observable is the lower-frequency cutting-regime envelope, which is why categorical/regime structure is recoverable while per-instruction numeric precision is not; the mechanism is physical, not only the sequence head's exposure bias. The 4 Hz rate is nevertheless *not* the sole numeric-recovery bottleneck. The per-row target formulation (Section 3.3) imposes a within-window row-identification ambiguity that is co-responsible for the numeric ceiling independently of sample rate (localized by Sections 6.4 and 6.8). A higher-rate stack would address the aliasing component; the autoregressive sequence head addresses the per-row component.

Spectral gap and sensor-stack upgrade path.

Two further methodological limitations are detailed in Supplementary Materials Section S34. First, we have not characterized the per-modality PSD/SNR of the 4 Hz envelope stream against a co-recorded raw stream, so we cannot attribute the numeric-tier ceiling between genuine envelope spectral loss and the sub-sample temporal aliasing of 77.7% of rows. The characterization therefore bounds what the 4 Hz envelope stack recovers, not the maximum recoverable from the underlying physical signal. Second, the numeric-precision ceiling is a property of the present stack, not the architecture. A higher-rate next-iteration stack (contact microphone $\geq$ 44.1 kHz, axis encoders / IMU / Hall-effect spindle sensors at $\geq$ 1 kHz) could close the numeric tier and the AR-mode gap while leaving the near-ceiling categorical recovery unchanged.

Window length.

The 256-sample window length was inherited from the encoder paper's classification configuration [22] to ensure consistent encoder inputs. A full window/stride sweep requires retraining the encoder for each (window, stride) pair and is therefore not separable on the present frozen encoder. It remains an open question tied to the encoder-retraining direction (Section 8.6).

8.3. Statistical Power Limitations

Per-class metrics for the command head are computed on small per-class supports in the test split: G0 (n = 86), G1 (n = 1,095), G2 (n = 475), G3 (n = 395), none (n = 5,886). A power analysis (`audit/extended_rigor_audits.json`, key `rec6_power_analysis`) computes the minimum detectable effect (MDE) at $\alpha = 0.05$, $\beta = 0.20$ for each class:

G0 ($n = 86$, baseline $\sim$0.40) MDE $\approx$ 18.8 pp; G1 ($n = 1{,}095$, baseline $\sim$0.94) MDE $\approx$ 2.3 pp; G2 ($n = 475$, baseline $\sim$0.50) MDE $\approx$ 8.0 pp; G3 ($n = 395$, baseline $\sim$0.30) MDE $\approx$ 8.4 pp; none ($n = 5{,}886$, baseline $\sim$0.95) MDE $\approx$ 0.9 pp; param-types X, Y, Z, and R MDE $\leq$ 1.5 pp (large supports). Differences below these sizes are not statistically distinguishable from sampling noise. "Not significant" on the long-tail classes (G0, G3) often means *underpowered*, not *equivalent*. Resolving these per-class differences would require substantially larger per-class supports than the present corpus provides.

Covariate-shift audit.

A per-field train-vs-test positive-support audit (`audit/extended_rigor_audits.json`, keys `rec7_covariate_shift_*`) flags only modest shifts ($<$6 pp) on already-low-support fields: per-row fold-1 Z $-$4.4 pp; full-window fold-1 F +3.2 pp and fold-5 F $-$5.3 pp; no flagged fields on the remaining folds. These shifts contribute to the cross-fold standard deviation in Table 4 but do not change the per-axis qualitative ranking.

8.4. Per-File Performance Variance

Slicing the 5-fold predictions by source file (n = 91) shows per-file command accuracy is concentrated, not heavy-tailed (0.953 $\pm$ 0.088; 10th percentile still 0.846). The categorical-head success is therefore not concentrated on a few memorizable files. AR token accuracy shows the expected wide per-file spread, tracking the per-operation-class breakdown (Section 6.5). The full per-file distribution is in Supplementary Section S10.3, together with the reconciliation of the unweighted per-file mean (0.953) against the per-fold-mean headline (0.8875 $\pm$ 0.0558), an aggregation difference rather than a prediction discrepancy.

8.5. Evaluation Limitations

Single-run per condition.

Each operation-class file represents a single execution. The metrics in Section 6 therefore conflate cross-condition variance (different operation classes) with cross-trial variance (rerunning the same operation). A designed multi-replicate corpus (multiple executions per condition) would support variance decomposition; the present corpus does not.

Decoder-only inference cost.

The decoder generates token-by-token autoregressively under a grammar mask. Per-row autoregressive decoding on the RTX A6000 is on the order of 30–80 ms per G-code line at the median row length (5–15 tokens, $K = 2{,}418$, FSM-masked greedy), well within the 250 ms inter-sample headroom of the 4 Hz acquisition cadence. Full-window mode ($\sim$1,000–1,400 tokens per sample) costs proportionally more and is not real-time on this hardware. A beam-search or non-autoregressive variant has not been evaluated; doing so would inform deployment-cost trade-offs (Section 7.2.0.1).

K-spectrum methodology confound: resolved.

The four-point vocabulary-cardinality sweep originally compared the headline $K = 2{,}418$ row (SS=0.5, dw=1.0) against three smaller-K variants trained under Design B (SS=0, dw=0). The smaller-K-vs-headline comparison was therefore a joint contrast of vocabulary cardinality and training schedule. A methodology-matched $K = 2{,}418$ control trained under SS=0/dw=0 (T3.1; Section 6.10.2) resolves the confound. Under matched methodology, $K = 2{,}418$ is the highest-performing variant on every metric (AR struct token 0.470 ± 0.034 vs. K=335's 0.429 ± 0.123 and the headline's 0.317 ± 0.116). The earlier +11.1 pp K=335 advantage over the headline was the SS/dw schedule change, not vocabulary cardinality. The K=335 "sweet spot" framing is accordingly retracted in this revision. The deployment-relevant configuration for AR structural recovery is the original $K = 2{,}418$ vocabulary with SS=0 and dw=0; the smaller-K family is now reported as a clean vocabulary-cardinality contrast within one methodology, with the deployment story carried by the matched-methodology control. Corpus generalization to other platforms and material classes is a separate open question. The K=2418/SS=0/dw=0 result requires cross-platform replication before it can carry forward (Section 8.6).

TF-numeric baseline reversal (future-work item, not a deployment recommendation).

The seq2seq baseline of Section 6.14 attains a higher teacher-forced numeric accuracy (0.754 ± 0.053) than the headline six-head digit-decomposition output (0.585 ± 0.033). The gap is attributable to the bucketization that makes the grammar mask tractable in the legacy-token head (Section 3.2). We flag this as a future-work item for architectural refinement of the numeric head, not as a deployment recommendation. Under autoregressive inference, the regime that matters for the post-hoc audit posture, the two architectures' numeric accuracies are within a fold of each other (0.100 vs. 0.104), so the TF-numeric reversal does not translate to an AR-mode deployment advantage for the baseline.

8.6. Future Directions

Four directions remain open. Each is an open problem rather than a scheduled experiment.

(a) Continuous-parameter data-collection campaign. A designed factorial corpus would vary feed rate (F), spindle speed (S), depth of cut, and material at a higher sampling rate (≥ 100 Hz, to lift the 4-Hz aliasing of 77.7% of rows), populating the F/S/I/J fields the present 4-Hz corpus cannot evaluate. The first experiment is a $3 \times 3 \times 3$ (feed, speed, depth) full factorial on the present platform with the higher-rate acquisition front end, scoped to recover feed-rate substitutions at the resolution the adversarial scenarios of Section 7.4 require.

(b) Encoder retraining with row-level supervision. The present fold-1 last-LSTM-layer fine-tune pilot (Section 4.6) is within fold-1 noise and underpowered to settle the encoder-bottleneck question. The deliverable artifact is a five-fold retrained encoder under matched preprocessing, with checkpoints released alongside this manuscript's, so that a reproducer can pair the retrained encoder with the present decoder and isolate the encoder-side residual.

(c) Window/stride re-optimization. This direction requires the encoder retraining of (b) and is therefore not separable on the present frozen encoder. The planned cell coverage is a 4×4 (window, stride) grid retrained from scratch per cell with the row-level-supervised encoder.

(d) Cross-platform validation. A second CNC platform (production-class VMC: cast-iron base, servo drive, flood coolant) and a second material class (aluminum or steel) with a matching sensor stack. The protocol is the present file-level five-fold characterization rerun on the new corpus. The deliverable is the cross-platform transfer matrix for the categorical-vs-numeric factorization.

9. Conclusion

This study characterizes which components of an executing G-code block are recoverable from multi-modal CNC sensor data and which are not. The characterization uses G-Code Decoder, a six-head Transformer decoder on a frozen multi-modal denoising encoder with per-row targets and grammar-constrained inference, under five-fold file-level cross-validation. Four findings follow, each detailed in the section cited:

(1) **Per-block categorical recovery** (command identity, axis presence, motion direction) is sensor-driven and TF/AR-invariant by construction (Section 6.2). The class-prior-controlled within-class increment over the operation-class modal predictor (Section 7.1.0.2) is the figure that survives conditioning the null on operation class. On an *unseen* operation class the increment does not transfer. Command recovery there collapses to the open-vocabulary floor of 0.213, below the class prior (Section 6.12).

(2) **End-to-end token/numeric** reproduction is at floor (autoregressive whole-sequence exact-match 0.026). Two decoder-side mechanisms bound it: exposure-bias mode-collapse in the autoregressive head (Section 7.3) and 4-Hz temporal aliasing of 77.7% of rows (Section 8.2). These bounds act on top of an information-theoretic floor that the companion limit paper [33] characterizes. On the float-epsilon-corrected retrain, the per-digit precision floor sits at the *thousandths* (10^{-3} inch, slot accuracy 0.42), the finest digit the CAM postprocessor emits. The ten-thousandths digit (10^{-4}) is near-uniform in the four-decimal source but is discarded by the V8 tokenizer's 0.001 grid. The tokenized digit is therefore a structural zero reproduced trivially, not an information-bearing recovery. Generative coordinate reconstruction is unsupported on this stack. The K-spectrum retraction and the matched-methodology T3.1 control are reported in Section 8.5.

(3) **Sensor-stack redundancy.** Under the frozen encoder, inference-time modality removal collapses command accuracy uniformly ($p = 0.998$ between modalities; Section 6.11). Encoder-retraining ablations (LOMO and LOSO; Section 7.2.0.1) show that no single sensing modality or sensor unit is individually decisive ($\Delta \leq 2.6$ pp). This redundancy enables a cost-reduced post-hoc audit installation.

(4) **Tamper detection** is largely prior-driven, with a sensor-conditioned lift of ≈ 8 pp over the corpus modal. Under the open-vocabulary leave-one-class-out regime detection collapses (command-swap FPR 0.52). The in-distribution $K = 10$ point at the measured $\rho = 0.42$ (FPR 0.10–0.18 at TPR ≈ 0.94–0.97) is an optimistic in-envelope ceiling (Section 7.4). That ceiling supports the intended deployment posture of **forensic-support post-hoc audit** of contested runs at human-in-the-loop $K = 10$ majority, *not* a validated real-time SOC detector or autonomous alerting. The audit posture is explicitly scoped to in-distribution operation classes, closed-world over the V8 vocabulary and the nine operation classes catalogued in Section 5.1. Open-world / novel-operation-class deployment is out of scope and deferred to future cross-platform replication (Section 8).

A from-scratch Transformer seq2seq baseline (Section 6.14) matches the headline command accuracy at teacher-forced inference but collapses at autoregressive inference. The headline numbers therefore stand. The architectural contribution of the frozen encoder + grammar-constrained decoder + structured heads is sharpened to AR deployment-mode inference, the regime that matters for the post-hoc audit posture this paper supports.

Closing the gap to full reconstruction is a sequence-head problem first. Whether the encoder is co-responsible cannot be settled by the present evidence. Section 4.6 reports only a single-fold, last-LSTM-layer fine-tune pilot, which is within fold-1 noise and underpowered as a structural claim; Supplementary Materials Section S21 restates this. Settling the encoder question requires full 5-fold encoder retraining with row-level supervision across the multi-modal projection and attention stack.

Standard exposure-bias remedies [70–74], a higher sample rate, encoder retraining with row-level supervision, and richer continuous-parameter data are the principal directions for the TRL 5–6 cross-platform target (Section 8). Cross-platform replication on a production-class VMC is the prerequisite for any deployment claim outside the single desktop CNC platform characterized here. The present **forensic-support post-hoc audit** posture is the boundary within which the operating points hold. None of these directions directly upgrades that posture to real-time SOC alerting. Closing the alert-budget gap requires a structurally different sensing stack and an open-vocabulary recoverability regime the present corpus does not support.

Appendix A Pointers to the Supplementary Materials

Specification addenda.

Three self-containment addenda are in the Supplementary Materials. The **coverage-repair split procedure** (Section S20) details how the file-level stratified k-fold splits are post-processed into the closed-vocabulary evaluation regime by swapping files until every test line also appears in its fold's training partner (0–3 swaps per fold; deterministic alphabetical tie-break; per-fold assignment released as `audit/file_fold_assignment.csv`). The **encoder specification addendum** (Section S21) restates the 8.6 M-parameter multi-modal denoising encoder's architecture and training, and quantifies the representation-reuse caveat: the worst-case class-label-leakage bound caps the leakage component of command accuracy at 0.572 (so the $+22$ pp class-prior-controlled increment of Section 7.1.0.2 is leakage-immune), and the leave-decoder-test-files-out LOMO-baseline check bounds the empirical reuse effect at $|\Delta| \leq 0.016$ (0.904 ± 0.039 vs. headline 0.888 ± 0.056). The **grammar tokenization addendum** (Section S22) gives the per-axis quantization precisions, the `NUM_a_b` bucket-index formula, and the float-epsilon-corrected round-to-scaled-integer encoding (categorical/token metrics drift $\leq$1.5 pp between encodings).

Replicability checklist.

Code, data DOIs, random-seed and determinism details, hardware/software versions, hyperparameter sweep ranges, statistical methodology, and inference-cost detail are in Supplementary Section S19.

Supplementary diagnostic analyses.

Supplementary Section S10 collects six corroborating diagnostics referenced from the body: test-time sensor-noise sensitivity (S10.1); nested shortcuts$\times$modality and pattern$\times$modality interaction probes (single-fold pilots; no interaction overturns the modality main effect; S10.2); per-file performance variance (S10.3); beam-width sensitivity (the TF$\rightarrow$AR gap is not closed by beam-3/5; S10.4); the output-position failure distribution (S10.5); and the full K-row/ρ/rule tamper-aggregation grid with the ROC-space view (S10.6).

Extended ablations.

The per-physical-sensor-unit LOSO study with encoder retraining (Supplementary Section S5), the nested 6×6 leave-one-(sensor,modality)-out retraining (Supplementary Section S6), and four single-fold diagnostic probes (Supplementary Section S8) corroborate the LOMO null. Their statistics are summarized in the body (Section 6.11.1, “Orthogonal probes”), and all single-fold pilots are excluded from quantitative claims.

Non-neural baseline comparison.

The full non-neural / lightweight-baseline comparison (the HGB tree-booster and the MLP probe of Section 4.5, both on the mean-pooled encoder embedding) is in Supplementary Materials Section S23, together with the per-task accuracy/macro-F1 table and the neural-seq2seq-baseline scoping note; the headline reading is in Section 6.1.

Appendix B Relationship to the Companion Contributions and Data Availability

This study is one of four related contributions and we state the division explicitly. (i) The encoder paper [22] establishes a multi-modal denoising sensor encoder and its operation-class representational competence (96.3% nine-class accuracy). (ii) A grammar paper [23] defines the hybrid G-code tokenization. (iii) *The present paper* addresses per-row, instruction-level G-code *recoverability*: the per-field structured-output evaluation, the metadata-versus-sensor and per-row-versus-full-window ablations, the leave-one-class-out generalization study, the exposure-bias / mode-collapse characterization, and the tamper-detection scoping. It takes the encoder and grammar as fixed and asks what row-level instruction information survives the sensor→latent map; it neither re-derives the encoder nor the grammar. (iv) A companion limit paper [33] develops the decoder-agnostic information-theoretic and physical recoverability limit — the per-(axis, slot, operation-class) source entropy of the CNC toolpath corpus, the per-axis sign and per-digit entropy decompositions, the float-epsilon numeric-target-encoding audit, and the 4 Hz sub-Nyquist floor — that bounds the per-field recoverability characterized here; the present paper diagnoses the recoverability phenomenon and cites the companion for the underlying limit. We acknowledge that, stripped of the companion contributions, the standalone methodological novelty is modest (a frozen embedding, a Transformer decoder, and a runtime grammar mask); the contribution of this paper is the characterization itself — including the negative results and the explicit in-distribution / open-vocabulary boundary — not a new model component. The encoder and grammar specification addenda (Supplementary Materials Sections S21–S22) fold the encoder and grammar specifications into the Supplementary Materials so that the work is independently assessable and reproducible.

Data and code availability.

The 120-file aligned corpus, the per-fold preprocessed arrays, the decoded prediction JSONs, and all audit artifacts referenced in this paper are released as described in the Data Availability Statement below. Nothing required to reproduce the results in this manuscript depends on the companion contributions. *Path convention:* all file paths cited (e.g., `audit/digit_entropy.json`, `src/miracle/model/digit_value_head.py`, `scripts/analysis/...`) are relative to the repository root unless explicitly noted; `outputs/decoder20260511/` is the project working directory for the artifacts referenced throughout.

Author Contributions: Conceptualization, S.S.E. and R.S.P.; methodology, S.S.E.; software, S.S.E.; validation, S.S.E.; formal analysis, S.S.E.; investigation, S.S.E.; resources, M.S.S.; data curation, S.S.E. and R.S.P.; writing—original draft, S.S.E.; writing—review and editing, R.S.P. and M.S.S.; visualization, S.S.E.; supervision, R.S.P. and M.S.S.; project administration, R.S.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The sensor datasets and analysis code generated during this study are publicly available at https://github.com/stephenseacuello/gcode_fingerprinting (accessed on 28 May 2026), at the tagged release `v1.0-submission` corresponding to the commit hash recorded in the repository's `CITATION.cff`. A pre-acceptance Zenodo DOI for the same tagged release is minted via the GitHub–Zenodo integration prior to submission and is listed in the repository's top-level `README` alongside the SHA256 manifest of the released checkpoints (per-fold encoder weights, headline decoder weights, and the T3.1 control). Source code for the G-Code Decoder decoder architecture, training and evaluation pipelines, audit-JSON regeneration scripts, and figure-generation scripts is released under the MIT License; the 120-file aligned corpus is released under CC BY 4.0.

Acknowledgments: The authors thank the Department of Industrial and Systems Engineering at the University of Rhode Island for providing laboratory facilities and computational resources.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

Abbreviations

The following abbreviations are used in this manuscript:

ANOVA, Analysis of Variance; AR, Autoregressive; AUC, Area Under the Curve; BH-FDR, Benjamini–Hochberg False Discovery Rate; CAM, Computer-Aided Manufacturing; CI, Confidence Interval; CNC, Computer Numerical Control; CV, Cross-Validation; dw, Digit-head Loss Weight; ECE, Expected Calibration Error; EMI, Electromagnetic Interference; FDM, Fused Deposition Modeling; FPR, False Positive Rate; FSM, Finite-State Machine; ICS, Industrial Control System; LOCO, Leave-One-Class-Out; LOMO, Leave-One-Modality-Out; LSTM, Long Short-Term Memory; LOSO, Leave-One-Sensor-Out; MCE, Maximum Calibration Error; MAE, Mean Absolute Error; MDE, Minimum Detectable Effect; MM-DTAE-LSTM, Multi-Modal Denoising Transformer Autoencoder with Long Short-Term Memory; NIST, National Institute of Standards and Technology; NLL, Negative Log-Likelihood; PCA, Principal Component Analysis; pp, Percentage Points; ROC, Receiver Operating Characteristic; RMS, Root Mean Square; SS, Scheduled Sampling; TCB, Trusted Computing Base; TF, Teacher-Forced; TPR, True Positive Rate.

1. National Institute of Standards and Technology. Guide to Operational Technology (OT) Security. Technical Report SP 800-82 Rev. 3, NIST, 2023.
2. Vogl, G.W. Research Needs for Cyberphysical Systems in Machining and Machine Tools: FMNet 2024 Report. Technical report, National Institute of Standards and Technology, 2024. Future of Manufacturing Network (FMNet) 2024 workshop report.
3. Sturm, L.D.; Williams, C.B.; Camelio, J.A.; White, J.; Parker, R. Cyber-physical vulnerabilities in additive manufacturing systems. International Solid Freeform Fabrication Symposium, 2014, pp. 951–963.
4. Yampolskiy, M.; King, W.E.; Gatlin, J.; Belikovetsky, S.; Brown, A.; Skjellum, A.; Elovici, Y. Security of Additive Manufacturing: Attack Taxonomy and Survey. *Additive Manufacturing* **2018**, *21*, 431–457.
5. Zhong, R.Y.; Xu, X.; Klotz, E.; Newman, S.T. Intelligent Manufacturing in the Context of Industry 4.0: A Review. *Engineering* **2017**, *3*, 616–630. doi:10.1016/J.ENG.2017.05.015.
6. Kimmell, J.C.; Orlyanchik, V.; Sturm, L.; Dawson, J.; Taylor, C. Detecting Control Injection Attacks Using Energy Data Anomalies in Computer Numerical Control Machining. *Critical Infrastructure Protection XVIII* **2025**, *725*, 43–63.
7. Teti, R.; Jemielniak, K.; O'Donnell, G.; Dornfeld, D. Advanced monitoring of machining operations. *CIRP Annals* **2010**, *59*, 717–739.
8. Serin, G.; Sener, B.; Ozbayoglu, A.M.; Unver, H.O. Review of tool condition monitoring in machining and opportunities for deep learning. *The International Journal of Advanced Manufacturing Technology* **2020**, *109*, 953–974.
9. Zhu, K.; Wong, Y.S.; Hong, G.S. Wavelet analysis of sensor signals for tool condition monitoring: A review and some new results. *International Journal of Machine Tools and Manufacture* **2009**, *49*, 537–553.

10. Mohanraj, T.; Shankar, S.; Rajasekar, R.; Sakthivel, N.; Pramanik, A. Tool condition monitoring techniques in milling process — a review. *Journal of Materials Research and Technology* **2020**, *9*, 1032–1042.
11. Wang, H.; Wang, S.; Sun, W.; Xiang, J. Multi-Sensor Signal Fusion for Tool Wear Condition Monitoring Using Denoising Transformer Auto-Encoder ResNet. *Journal of Manufacturing Processes* **2024**, *124*, 1227–1238.
12. Wang, J.; Ma, Y.; Zhang, L.; Gao, R.X.; Wu, D. Deep learning for smart manufacturing: Methods and applications. *Journal of Manufacturing Systems* **2018**, *48*, 144–156. doi:10.1016/j.jmsy.2018.01.003.
13. Zhao, R.; Yan, R.; Chen, Z.; Mao, K.; Wang, P.; Gao, R.X. Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing* **2019**, *115*, 213–237. doi:10.1016/j.ymssp.2018.05.050.
14. Stathatos, E.; Tzimas, E.; Benardos, P.; Vosniakos, G.C. Convolutional Neural Networks for Raw Signal Classification in CNC Turning Process Monitoring. *Sensors* **2024**, *24*, 1390.
15. Coelho, P.J.; Athar, A.; Mozumder, M.A.I.; Ali, S.; Kim, H.C. Deep Learning-Based Anomaly Detection Using One-Dimensional Convolutional Neural Networks (1D CNN) in Machine Centers (MCT) and Computer Numerical Control (CNC) Machines. *PeerJ Computer Science* **2024**, *10*, e2389. doi:10.7717/peerj-cs.2389.
16. Al Faruque, M.A.; Chhetri, S.R.; Wan, J.; Canedo, A. Acoustic Side-Channel Attacks on Additive Manufacturing Systems. Proceedings of the 7th ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS), 2016, pp. 1–10. doi:10.1109/ICCPS.2016.7479068.
17. Song, C.; Lin, F.; Ba, Z.; Ren, K.; Zhou, C.; Xu, W. My Smartphone Knows What You Print: Exploring Smartphone-Based Side-Channel Attacks Against 3D Printers. Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016, pp. 895–907. doi:10.1145/2976749.2978300.
18. Chhetri, S.R.; Canedo, A.; Al Faruque, M.A. KCAD: Kinetic Cyber-Attack Detection Method for Cyber-Physical Additive Manufacturing Systems. 35th IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2016, pp. 1–8. doi:10.1145/2966986.2967050.
19. Chhetri, S.R.; Canedo, A.; Al Faruque, M.A. Confidentiality Breach Through Acoustic Side-Channel in Cyber-Physical Additive Manufacturing Systems. *ACM Transactions on Cyber-Physical Systems* **2017**, *2*, 3:1–3:25. doi:10.1145/3078622.
20. Jamarani, A.; Tu, Y.; Hei, X. Practitioner Paper: Decoding Intellectual Property: Acoustic and Magnetic Side-Channel Attack on a 3D Printer. Security and Privacy in Cyber-Physical Systems and Smart Vehicles (SmartSP 2024). Springer, 2024, Vol. 622, *LNICST*, pp. 33–52. doi:10.1007/978-3-031-93354-7_3.
21. Chattopadhyay, T.; Ceschin, F.; Garza, M.E.; Zyunkin, D.; Chhotaray, A.; Stebner, A.P.; Zonouz, S.; Beyah, R. One Video to Steal Them All: 3D-Printing IP Theft through Optical Side-Channels. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2025. doi:10.1145/3719027.3744837.
22. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. Multimodal Sensor-Fusion Deep Learning for CNC Toolpath and Condition Classification: A Cross-Validated Ablation Study. *Machines* **2026**. Under review; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
23. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. G-Code Grammar for LLMs: Hierarchical Tokenization of CNC Machine Instructions, 2026. In preparation; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
24. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. Advances in Neural Information Processing Systems (NeurIPS), 2014, Vol. 27.
25. Bahdanau, D.; Cho, K.; Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. 3rd International Conference on Learning Representations (ICLR), 2015.
26. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 2017, Vol. 30.
27. Graves, A.; Mohamed, A.r.; Hinton, G. Speech Recognition with Deep Recurrent Neural Networks. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2013, pp. 6645–6649.
28. Meyes, R.; Lu, M.; de Puiseau, C.W.; Meisen, T. Ablation Studies in Artificial Neural Networks. *arXiv preprint arXiv:1901.08644* **2019**.

29. Jia, F.; Lei, Y.; Lin, J.; Zhou, X.; Lu, N. Deep neural networks: A promising tool for fault characteristic mining and intelligent diagnosis of rotating machinery with massive data. *Mechanical Systems and Signal Processing* **2016**, *72–73*, 303–315. doi:10.1016/j.ymssp.2015.10.025.
30. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. Proceedings of EMNLP, 2021.
31. Willard, B.T.; Louf, R. Efficient Guided Generation for Large Language Models. *arXiv preprint arXiv:2307.09702* **2023**.
32. Dong, Y.; Ruan, C.F.; Cai, Y.; Lai, R.; Xu, Z.; Zhao, Y.; Chen, T. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. Proceedings of Machine Learning and Systems (MLSys), 2025.
33. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. Source-Entropy-Correlated Recoverability of CNC Toolpath Coordinates under Low-Rate Sensing: Quantization Structure, an Operation-Class Ordering, and the 4 Hz Sub-Nyquist Floor. *Entropy* **2026**. Companion paper.
34. Janssens, O.; Slavkovikj, V.; Vervisch, B.; Stockman, K.; Loccufier, M.; Verstockt, S.; Van de Walle, R.; Van Hoecke, S. Convolutional Neural Network Based Fault Detection for Rotating Machinery. *Journal of Sound and Vibration* **2016**, *377*, 331–345. doi:10.1016/j.jsv.2016.05.027.
35. Bhandari, B. Comparative Study of Popular Deep Learning Models for Machining Roughness Classification Using Sound and Force Signals. *Micromachines* **2021**, *12*, 1484. doi:10.3390/mi12121484.
36. Lahat, D.; Adali, T.; Jutten, C. Multimodal Data Fusion: An Overview of Methods, Challenges, and Prospects. *Proceedings of the IEEE* **2015**, *103*, 1449–1477. doi:10.1109/JPROC.2015.2460697.
37. Ramachandram, D.; Taylor, G.W. Deep Multimodal Learning: A Survey on Recent Advances and Trends. *IEEE Signal Processing Magazine* **2017**, *34*, 96–108. doi:10.1109/MSP.2017.2738401.
38. Cao, L.; Li, J.; Zhang, L.; Luo, S.; Li, M.; Huang, X. Cross-attention-based multi-sensing signals fusion for penetration state monitoring during laser welding of aluminum alloy. *Knowledge-Based Systems* **2023**, *261*, 110212. doi:10.1016/j.knosys.2022.110212.
39. Tsanousa, A.; Bektsis, E.; Kyriakopoulos, C.; Gómez González, A.; Leturiondo, U.; Gialampoukidis, I.; Karakostas, A.; Vrochidis, S.; Kompatsiaris, I. A Review of Multisensor Data Fusion Solutions in Smart Manufacturing: Systems and Trends. *Sensors* **2022**, *22*, 1734. doi:10.3390/s22051734.
40. Bergs, T.; Gierlings, S.; Auerbach, T.; Klink, A.; Schraknepper, D.; Augspurger, T. The Concept of Digital Twin and Digital Shadow in Manufacturing. *Procedia CIRP* **2021**, *101*, 81–84.
41. Wen, Q.; Zhou, T.; Zhang, C.; Chen, W.; Ma, Z.; Yan, J.; Sun, L. Transformers in Time Series: A Survey. Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI), 2023, pp. 6778–6786. doi:10.24963/ijcai.2023/759.
42. Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; Zeng, A. Code as Policies: Language Model Programs for Embodied Control. IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 9493–9500.
43. Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Computation* **1997**, *9*, 1735–1780. doi:10.1162/neco.1997.9.8.1735.
44. Malhotra, P.; Vig, L.; Shroff, G.; Agarwal, P. Long Short Term Memory Networks for Anomaly Detection in Time Series. 23rd European Symposium on Artificial Neural Networks (ESANN), 2015, pp. 89–94.
45. Vincent, P.; Larochelle, H.; Lajoie, I.; Bengio, Y.; Manzagol, P.A. Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion. *Journal of Machine Learning Research* **2010**, *11*, 3371–3408.
46. Poesia, G.; Polozov, O.; Le, V.; Tiwari, A.; Soares, G.; Meek, C.; Gulwani, S. Synchromesh: Reliable Code Generation from Pre-trained Language Models. International Conference on Learning Representations (ICLR), 2022.
47. Beurer-Kellner, L.; Fischer, M.; Vechev, M. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages (PACMPL), OOPSLA* **2023**, *7*, 1946–1969.
48. Geng, S.; Josifoski, M.; Peyrard, M.; West, R. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 10932–10952.

49. Jignasu, A.; Marshall, K.; Ganapathysubramanian, B.; Balu, A.; Hegde, C.; Krishnamurthy, A. Towards Foundational AI Models for Additive Manufacturing: Language Models for G-Code Debugging, Manipulation, and Comprehension. *arXiv preprint arXiv:2309.02465* **2023**.
50. Abdelaal, M.; Lokadjaja, S.; Engert, G. GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback. Proceedings of the 21st Conference on Database Systems for Business, Technology and Web (BTW), Industrial Track, 2025.
51. Wang, Z.; Jadhav, Y.; Pak, P.; Barati Farimani, A. Image2Gcode: Image-to-G-code Generation for Additive Manufacturing Using Diffusion-Transformer Model. *arXiv preprint arXiv:2511.20636* **2025**.
52. Jeon, J.; Sim, Y.; Lee, H.; Han, C.; Yun, D.; Kim, E.; Nagendra, S.L.; Jun, M.B.; Kim, Y.; Lee, S.W.; Lee, J. ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation. *Journal of Manufacturing Systems* **2025**, *79*, 504–514.
53. Zonouz, S.; Rrushi, J.; McLaughlin, S. Detecting Industrial Control Malware Using Automated PLC Code Analytics. *IEEE Security & Privacy* **2014**, *12*, 40–47. doi:10.1109/MSP.2014.113.
54. Belikovetsky, S.; Yampolskiy, M.; Toh, J.; Gatlin, J.; Elovici, Y. dr0wned – Cyber-Physical Attack with Additive Manufacturing. 11th USENIX Workshop on Offensive Technologies (WOOT), 2017.
55. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; others. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **2011**, *12*, 2825–2830.
56. The MITRE Corporation. MITRE ATT&CK for ICS. https://attack.mitre.org/matrices/ics/, 2024. Techniques T0843 Program Download, T0836 Modify Parameter, T0856 Spoof Reporting Message, T0855 Unauthorized Command Message, T0862 Supply Chain Compromise.
57. National Institute of Standards and Technology. The NIST Cybersecurity Framework (CSF) 2.0. NIST Cybersecurity White Paper, 2024. doi:10.6028/NIST.CSWP.29.
58. Ba, J.L.; Kiros, J.R.; Hinton, G.E. Layer Normalization. *arXiv preprint arXiv:1607.06450* **2016**.
59. Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* **2014**, *15*, 1929–1958.
60. Bengio, S.; Vinyals, O.; Jaitly, N.; Shazeer, N. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. Advances in Neural Information Processing Systems (NeurIPS), 2015, Vol. 28.
61. Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. 7th International Conference on Learning Representations (ICLR), 2019.
62. Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; Wojna, Z. Rethinking the Inception Architecture for Computer Vision. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2818–2826. doi:10.1109/CVPR.2016.308.
63. Lin, T.Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. Focal Loss for Dense Object Detection. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988.
64. Virtanen, P.; Gommers, R.; Oliphant, T.E.; Haberland, M.; Reddy, T.; Cournapeau, D.; Burovski, E.; Peterson, P.; Weckesser, W.; Bright, J.; others. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* **2020**, *17*, 261–272. doi:10.1038/s41592-019-0686-2.
65. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.
66. International Organization for Standardization. ISO 14649-1:2003 Industrial automation systems and integration — Physical device control — Data model for computerized numerical controllers — Part 1: Overview and fundamental principles (STEP-NC). Technical report, ISO, 2003.
67. Wells, L.J.; Camelio, J.A.; Williams, C.B.; White, J. Cyber-physical security challenges in manufacturing systems. *Manufacturing Letters* **2014**, *2*, 74–77. doi:10.1016/j.mfglet.2014.01.005.
68. Hanley, J.A.; McNeil, B.J. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology* **1982**, *143*, 29–36. doi:10.1148/radiology.143.1.7063747.
69. Cichonski, P.; Millar, T.; Grance, T.; Scarfone, K. NIST SP 800-61 Rev. 2: Computer Security Incident Handling Guide. Technical report, National Institute of Standards and Technology, 2012. doi:10.6028/NIST.SP.800-61r2.
70. He, T.; Zhang, J.; Zhou, Z.; Glass, J. Quantifying Exposure Bias for Neural Language Generation. *arXiv preprint arXiv:1905.10617* **2019**.

71. Ott, M.; Auli, M.; Grangier, D.; Ranzato, M. Analyzing Uncertainty in Neural Machine Translation. Proceedings of the 35th International Conference on Machine Learning (ICML), 2018, pp. 3956–3965.
72. Ranzato, M.; Chopra, S.; Auli, M.; Zaremba, W. Sequence level training with recurrent neural networks. International Conference on Learning Representations (ICLR), 2016.
73. Holtzman, A.; Buys, J.; Du, L.; Forbes, M.; Choi, Y. The curious case of neural text degeneration. International Conference on Learning Representations (ICLR), 2020.
74. Welleck, S.; Kulikov, I.; Roller, S.; Dinan, E.; Cho, K.; Weston, J. Neural text generation with unlikelihood training. International Conference on Learning Representations (ICLR), 2020.